

Centro Universitário UniFECAF

Curso de Engenharia da Computação

CONSTRUINDO UM PROJETO ÁGIL NO GITHUB: DA GESTÃO AO CONTROLE DE QUALIDADE

RELATÓRIO EXPLICATIVO

Adriano Mantoan

RA:175928

São Paulo

Julho de 2026

Sumário

1. Introdução	1
2. Objetivo	2
2.1 Objetivo Geral.....	2
2.2 Objetivos Específicos	2
3. Ferramentas Utilizadas	2
4. Desenvolvimento das Atividades	4
4.1 Levantamento dos Requisitos e Planejamento do Projeto	4
5. Dificuldades Encontradas	28
6. Análise dos Resultados.....	29
7. Conclusão	31
8. Referências	32

1. Introdução

Quando comecei este projeto, imaginei que o maior desafio seria desenvolver um sistema utilizando Python e Flask. Porém, conforme o trabalho foi evoluindo, percebi que a programação era apenas uma parte do processo. Ao longo da disciplina fui entendendo que a Engenharia de Software envolve planejamento, organização das atividades, documentação, testes e controle das alterações realizadas durante o desenvolvimento.

A proposta da atividade foi desenvolver um sistema de gerenciamento de tarefas para uma empresa fictícia chamada **TechFlow Solutions**. A partir dessa ideia, comecei organizando os requisitos e planejando como cada funcionalidade seria implementada. Para acompanhar essa organização, utilizei um quadro Kanban no GitHub Projects, onde registrei todas as atividades desde o início até a conclusão do projeto.

O desenvolvimento foi realizado utilizando Python com o framework Flask, banco de dados SQLite, HTML e CSS para a interface e Git para o controle de versões. Durante o projeto também tive meu primeiro contato mais aprofundado com ferramentas como GitHub Actions e PyTest, que permitiram automatizar os testes da aplicação e compreender melhor como funciona uma rotina de integração contínua.

Nem todas as etapas aconteceram exatamente como haviam sido planejadas. Em alguns momentos precisei pesquisar novas soluções, corrigir erros e até alterar o escopo inicial do sistema para incluir uma funcionalidade de prioridade das tarefas. Apesar das dificuldades, essas mudanças ajudaram a tornar o projeto mais completo e me fizeram compreender melhor como as alterações fazem parte do desenvolvimento de software.

Ao concluir este trabalho, percebo que consegui aplicar, na prática, vários conceitos estudados durante a disciplina. Mais do que entregar um sistema funcionando, este projeto representou uma oportunidade de aprender como um software é planejado, desenvolvido, testado e documentado de forma organizada.

2. Objetivo

2.1 Objetivo Geral

O principal objetivo deste projeto foi desenvolver uma aplicação web de gerenciamento de tarefas utilizando os conhecimentos adquiridos na disciplina de Engenharia de Software. Além da construção do sistema, busquei compreender como as etapas de planejamento, modelagem, desenvolvimento, testes e documentação se relacionam durante a execução de um projeto de software.

2.2 Objetivos Específicos

Durante o desenvolvimento deste trabalho procurei alcançar os seguintes objetivos:

- compreender os requisitos propostos para o sistema antes de iniciar o desenvolvimento;
- organizar as atividades utilizando um quadro Kanban no GitHub Projects;
- elaborar os diagramas UML para representar a estrutura e o funcionamento do sistema;
- desenvolver uma aplicação web utilizando Python, Flask e SQLite;
- implementar funcionalidades de cadastro, edição, exclusão, pesquisa e atualização do status das tarefas;
- registrar e documentar uma mudança de escopo ocorrida durante o desenvolvimento do projeto;
- aprender a utilizar o Git e o GitHub para controle de versões;
- desenvolver testes automatizados utilizando PyTest;
- configurar o GitHub Actions para executar automaticamente os testes da aplicação;
- documentar todas as etapas do desenvolvimento por meio deste relatório técnico.

3. Ferramentas Utilizadas

Durante o desenvolvimento do projeto utilizei diferentes ferramentas que contribuíram para cada etapa do trabalho, desde a programação da aplicação até o controle de versão, organização das atividades e realização dos testes automatizados. A seguir, apresento as principais ferramentas utilizadas e sua finalidade dentro do projeto.

Visual Studio Code (VS Code)

Foi o ambiente de desenvolvimento utilizado para escrever, organizar e editar todo o código da aplicação. Também utilizei o terminal integrado do VS Code para executar comandos do Git, instalar bibliotecas, iniciar o servidor Flask e realizar os testes automatizados.

Python

Foi a linguagem de programação escolhida para desenvolver toda a lógica do sistema. Durante o projeto utilizei Python para implementar as funcionalidades responsáveis pelo cadastro, edição, exclusão, pesquisa, alteração de status e controle das tarefas.

Flask

O Flask foi utilizado como framework para desenvolvimento da aplicação web. Ele permitiu criar as rotas do sistema, processar as requisições dos usuários e integrar a aplicação com as páginas desenvolvidas em HTML.

SQLite

Foi utilizado como banco de dados da aplicação. Sua principal função foi armazenar todas as informações das tarefas cadastradas, garantindo que os dados permanecessem disponíveis mesmo após o encerramento da aplicação.

HTML5 e CSS3

Foram utilizados para desenvolver a interface gráfica do sistema. O HTML foi responsável pela estrutura das páginas, enquanto o CSS permitiu organizar o layout e melhorar a aparência da aplicação.

Git e GitHub

Foram utilizados para realizar o controle de versão do projeto. Durante o desenvolvimento registrei diversas alterações por meio de commits, permitindo acompanhar a evolução da aplicação e manter todo o código armazenado no repositório remoto.

GitHub Projects

Foi utilizado para organizar as atividades do projeto utilizando a metodologia Kanban. Nele foram registradas todas as etapas do desenvolvimento, permitindo acompanhar quais atividades estavam pendentes, em andamento ou concluídas.

PyTest

Foi utilizado para desenvolver os testes automatizados da aplicação. Esses testes permitiram verificar se as principais funcionalidades continuavam funcionando corretamente após cada alteração realizada no sistema.

GitHub Actions

Foi utilizado para automatizar a execução dos testes sempre que uma nova atualização era enviada ao repositório. Dessa forma, foi possível verificar automaticamente se o projeto permanecia funcionando corretamente após cada commit.

Draw.io

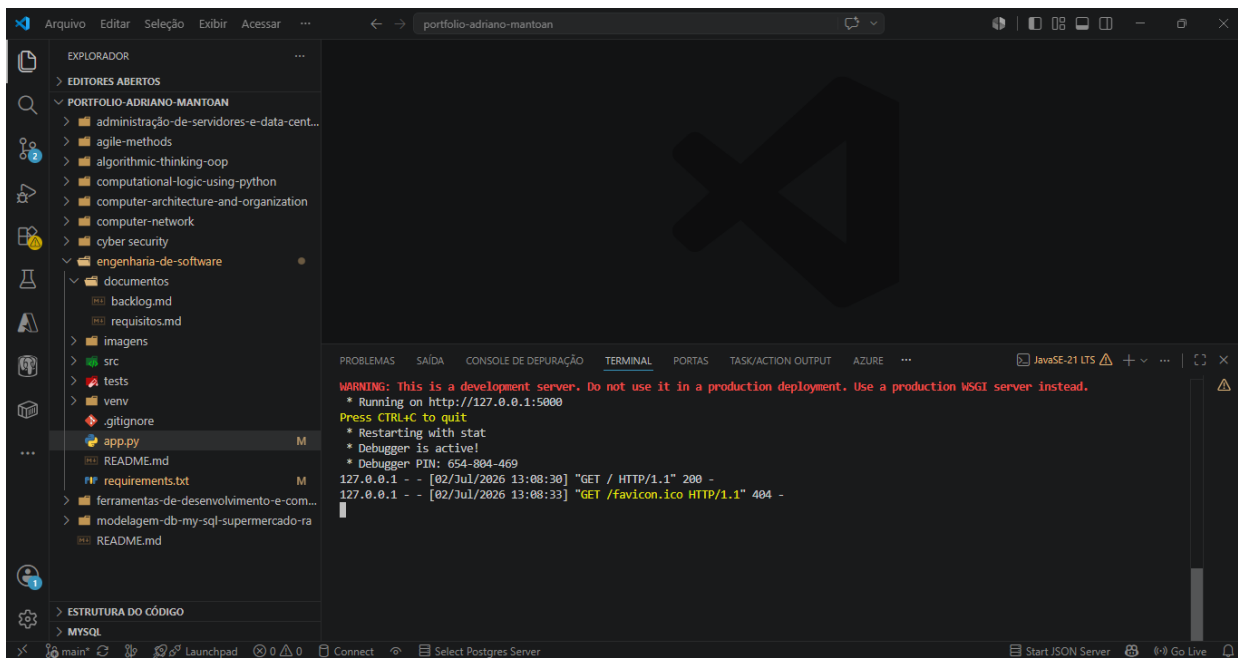
Foi utilizado para elaborar os Diagramas de Casos de Uso e de Classes, representando graficamente a estrutura e o funcionamento do sistema antes da implementação das funcionalidades.

4. Desenvolvimento das Atividades

4.1 Levantamento dos Requisitos e Planejamento do Projeto

Antes de iniciar a programação da aplicação, foi necessário compreender os requisitos apresentados na proposta da atividade. Essa etapa foi importante para definir quais funcionalidades fariam parte do sistema e como o desenvolvimento seria organizado ao longo do projeto.

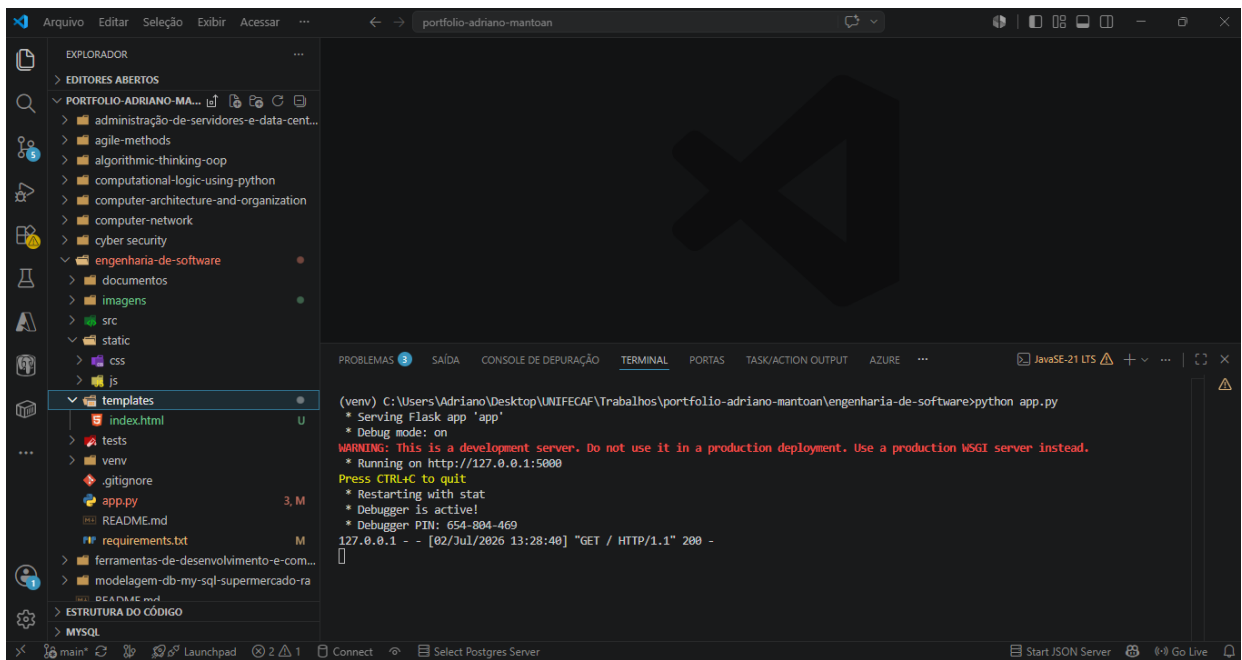
Figura 1 – Estrutura inicial do projeto TechFlow Task Manager



Fonte: Elaborado pelo autor (2026).

Nesta etapa iniciei a criação do projeto TechFlow Task Manager no Visual Studio Code. Organizei a estrutura inicial do diretório onde todos os arquivos do sistema seriam desenvolvidos. Também preparei o ambiente de trabalho para facilitar a implementação das funcionalidades previstas no Product Backlog, mantendo uma organização que permitisse futuras expansões do projeto.

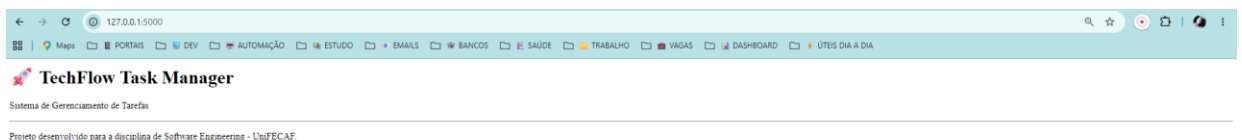
Figura 2 – Organização das pastas da aplicação Flask



Fonte: Elaborado pelo autor (2026).

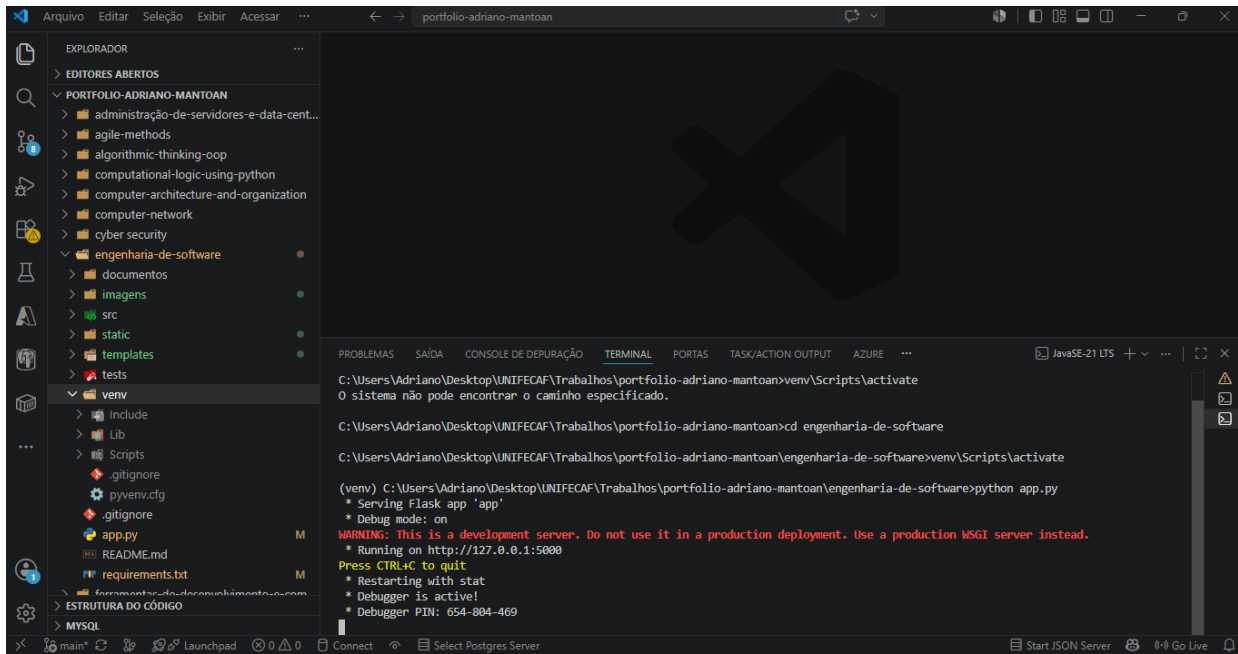
Após criar a estrutura inicial, organizei as pastas conforme o padrão utilizado pelo framework Flask. Foram criadas as pastas templates, responsável pelas páginas HTML, e static, destinada aos arquivos CSS e JavaScript. Essa organização tornou o desenvolvimento mais limpo e facilitou a manutenção do código durante todo o projeto.

Figura 3 – Primeira execução da aplicação Flask



Com a estrutura pronta, executei a aplicação pela primeira vez utilizando o servidor interno do Flask. Essa etapa foi importante para verificar se todas as configurações estavam funcionando corretamente antes de iniciar a implementação das funcionalidades do sistema.

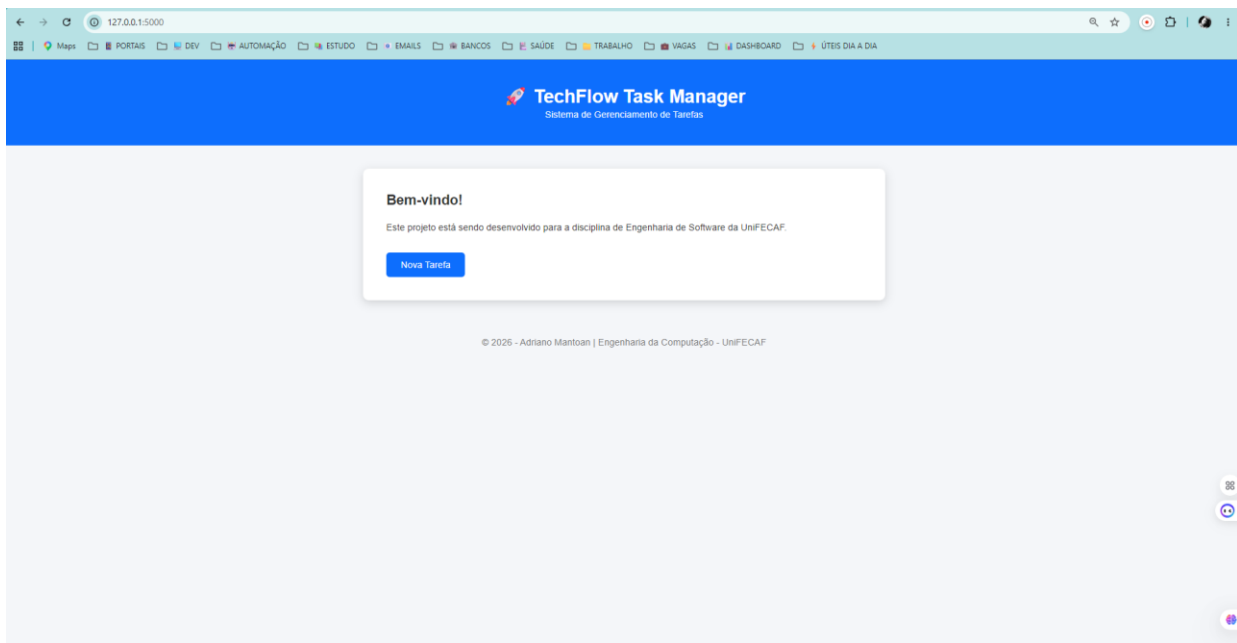
Figura 4 – Tela inicial da aplicação



Fonte: Elaborado pelo autor (2026).

Nesta etapa desenvolvi a primeira interface do sistema. A tela inicial foi criada para apresentar o nome do projeto, uma breve descrição e confirmar que a aplicação estava sendo carregada corretamente no navegador.

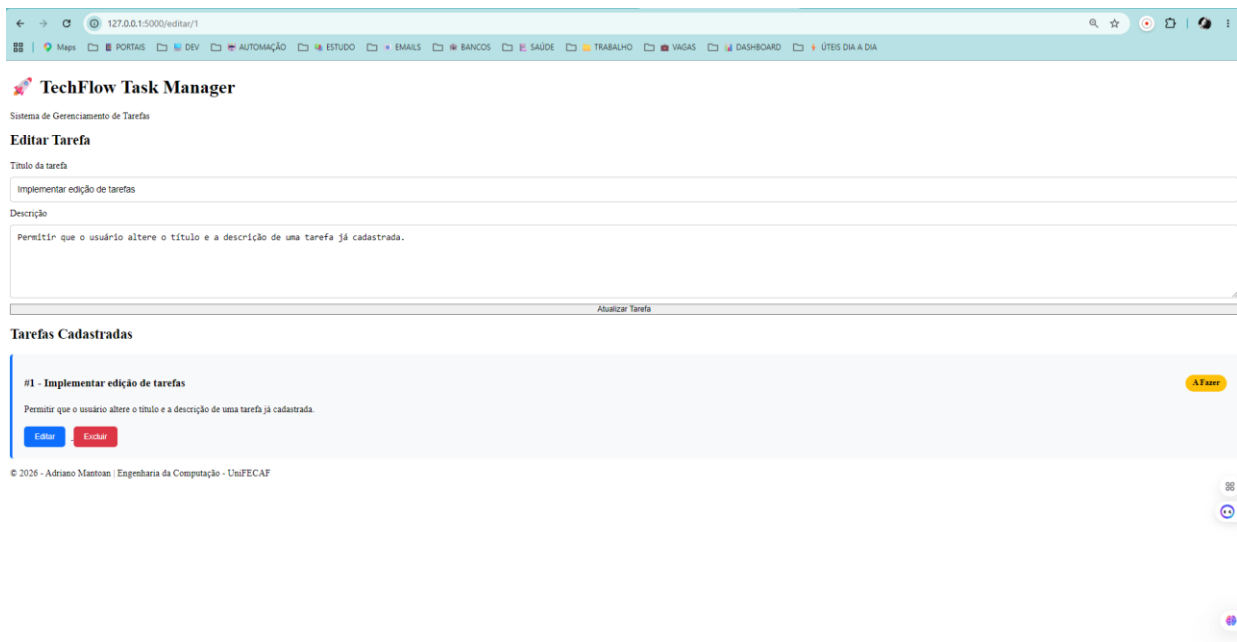
Figura 5 – Desenvolvimento da interface principal



Fonte: Elaborado pelo autor (2026).

Após validar a página inicial, iniciei o desenvolvimento da tela principal responsável pelo gerenciamento das tarefas. Nela foram disponibilizados os campos para cadastro das atividades e a área destinada à exibição das tarefas registradas.

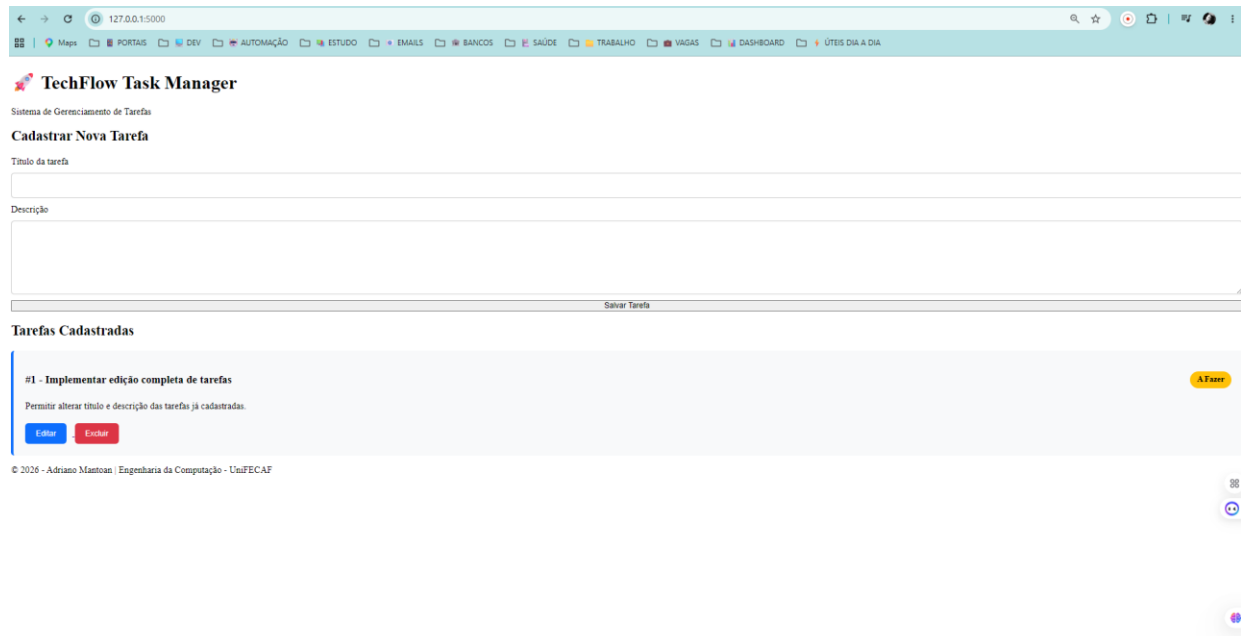
Figura 6 – Cadastro da primeira tarefa



Fonte: Elaborado pelo autor (2026).

Nesta etapa implementei a funcionalidade de cadastro de tarefas. Foi possível registrar um título e uma descrição, permitindo armazenar as informações que posteriormente seriam utilizadas para acompanhamento do andamento das atividades.

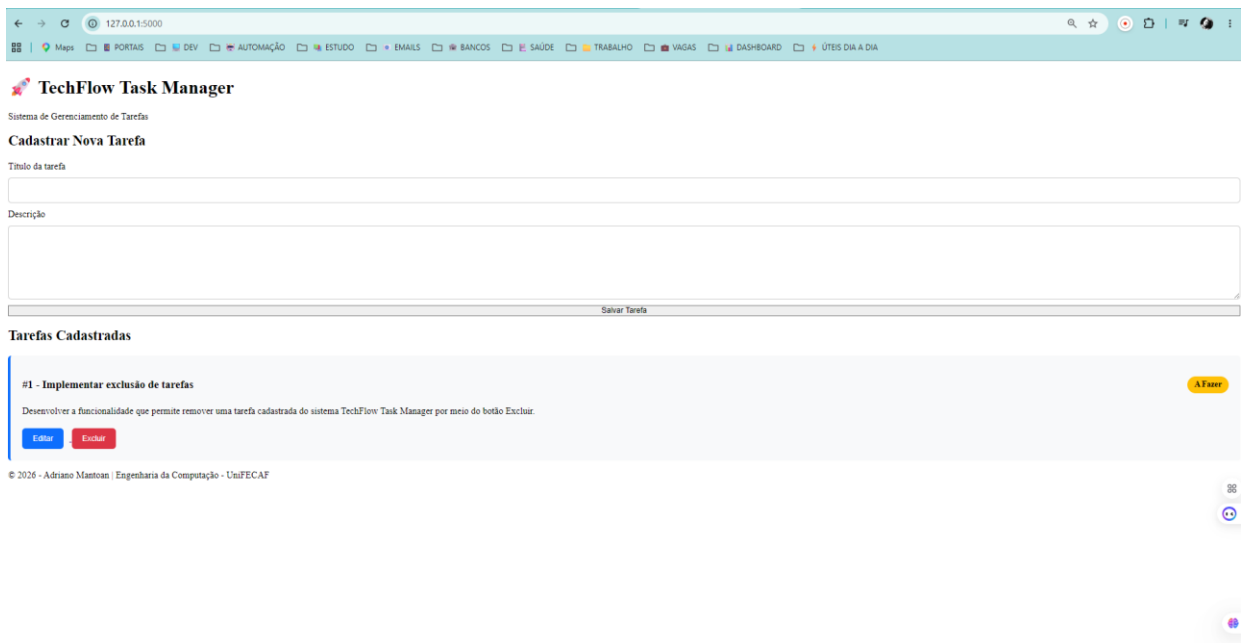
Figura 7 – Listagem das tarefas cadastradas



Fonte: Elaborado pelo autor (2026).

Depois da implementação do cadastro, desenvolvi a listagem das tarefas cadastradas. Cada tarefa passou a ser exibida em formato de cartão, contendo título, descrição e os botões necessários para edição, exclusão e alteração de status.

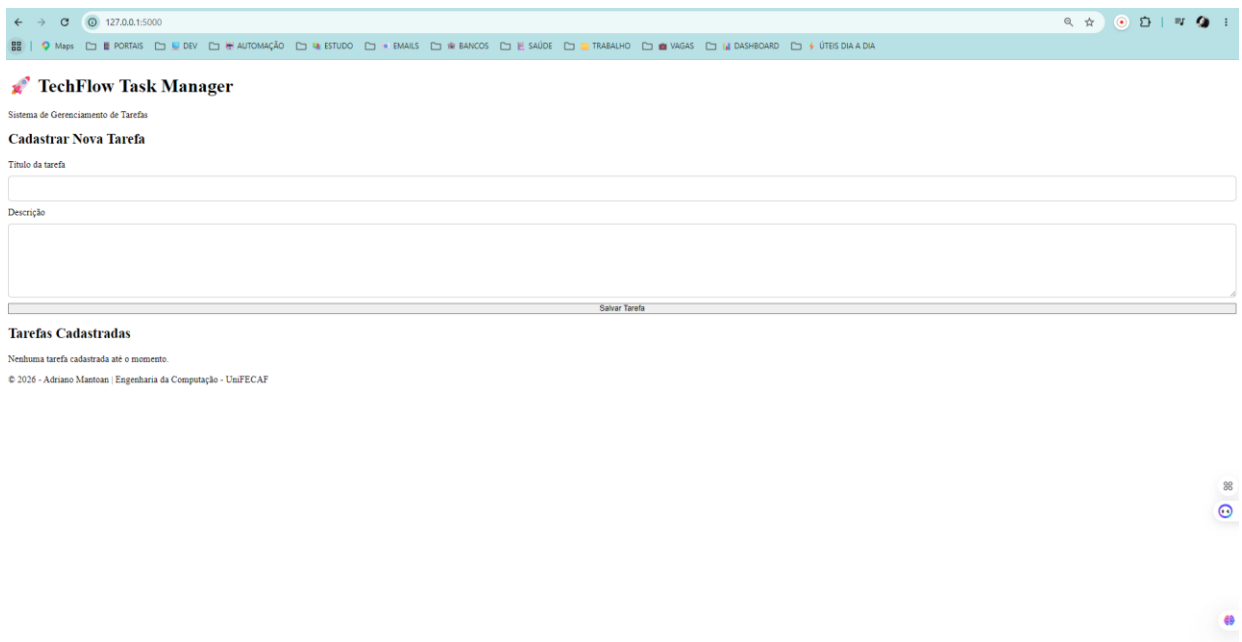
Figura 8 – Implementação da pesquisa de tarefas



Fonte: Elaborado pelo autor (2026).

Com a listagem funcionando, adicionei o recurso de pesquisa, permitindo localizar tarefas pelo título ou descrição. Essa funcionalidade facilita a localização de atividades quando existe uma quantidade maior de registros cadastrados.

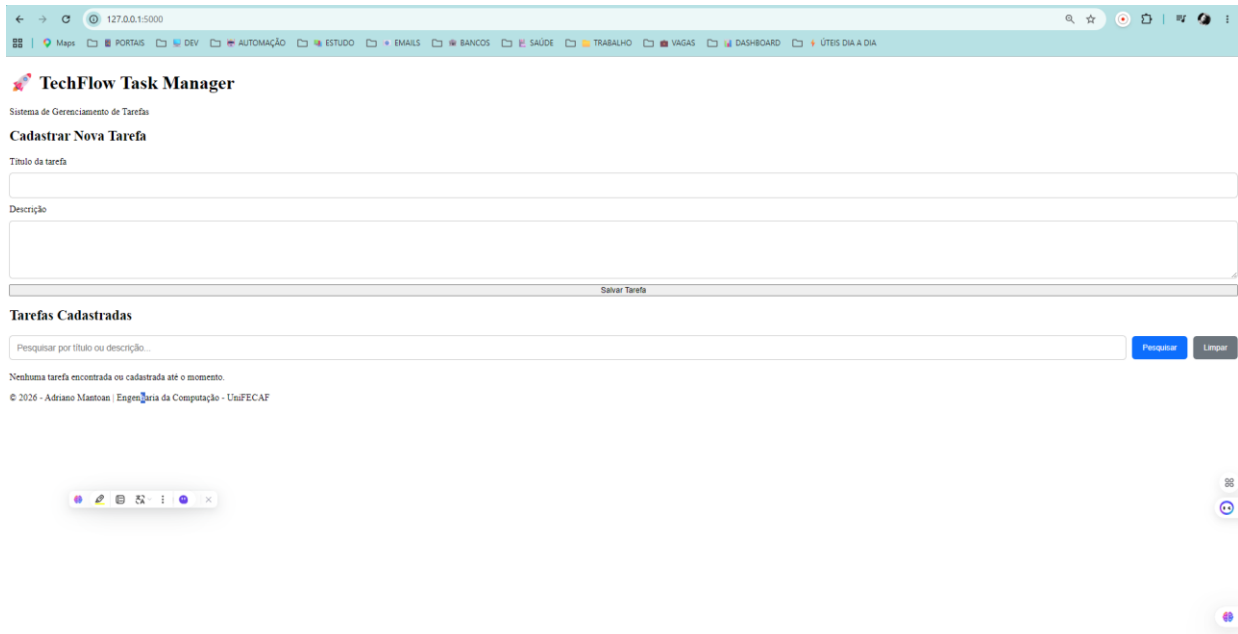
Figura 9 – Resultado da pesquisa por título



Fonte: Elaborado pelo autor (2026).

Após implementar a pesquisa, realizei testes utilizando palavras presentes no título das tarefas. O sistema apresentou apenas os registros compatíveis com o termo informado, demonstrando o funcionamento correto da funcionalidade.

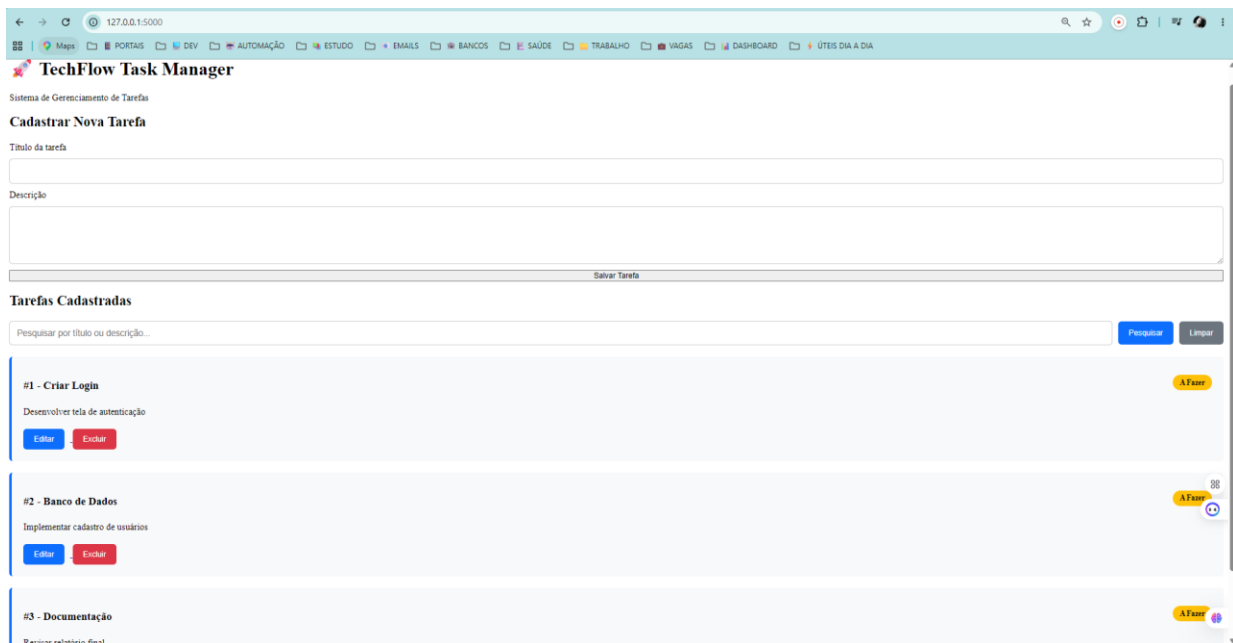
Figura 10 – Resultado da pesquisa por descrição



Fonte: Elaborado pelo autor (2026).

Também foram realizados testes utilizando palavras existentes na descrição das tarefas. Os resultados confirmaram que a pesquisa funciona tanto pelo título quanto pelo conteúdo descritivo.

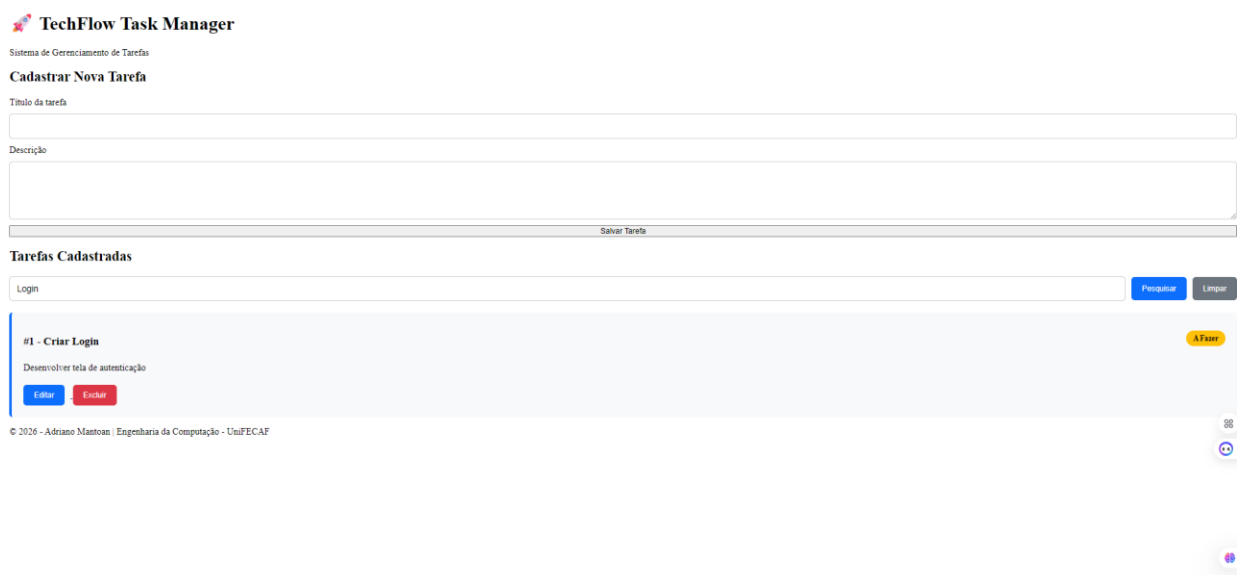
Figura 11 – Organização das tarefas no GitHub Projects



Fonte: Elaborado pelo autor (2026).

Durante o desenvolvimento utilizei o GitHub Projects como quadro Kanban para acompanhar a evolução do projeto. As tarefas foram distribuídas entre as colunas "A Fazer", "Em Progresso" e "Concluído", permitindo controlar todas as etapas de desenvolvimento.

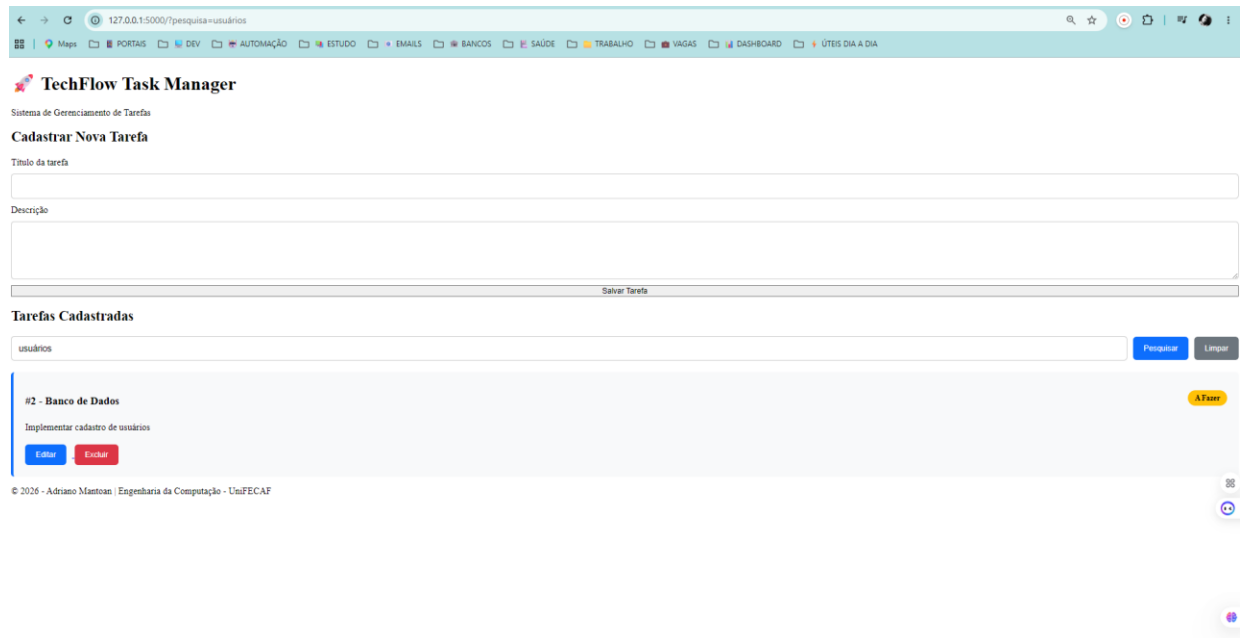
Figura 12 – Implementação da alteração de status



Fonte: Elaborado pelo autor (2026).

Nesta etapa desenvolvi a funcionalidade responsável por alterar o status das tarefas. O usuário passou a selecionar diretamente o andamento de cada atividade através de uma lista de opções disponível em cada cartão.

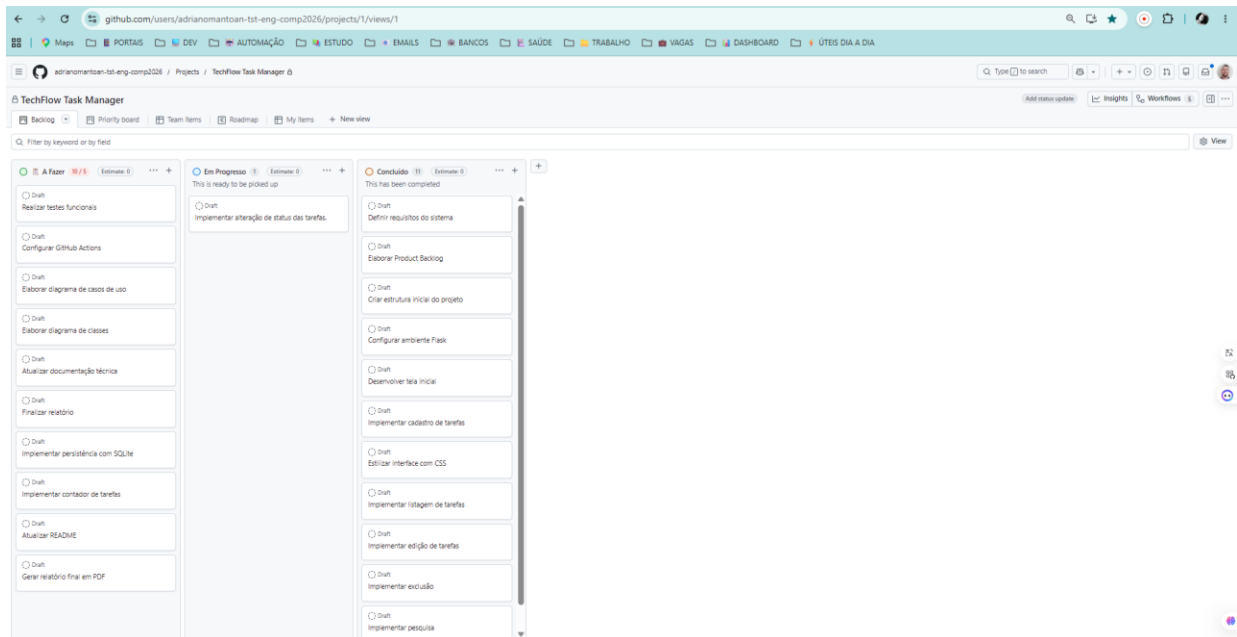
Figura 13 – Alteração para o status "Em Andamento"



Fonte: Elaborado pelo autor (2026).

Após implementar a funcionalidade de alteração de status, realizei testes alterando uma tarefa para o estado Em Andamento, verificando se a atualização era refletida corretamente na interface.

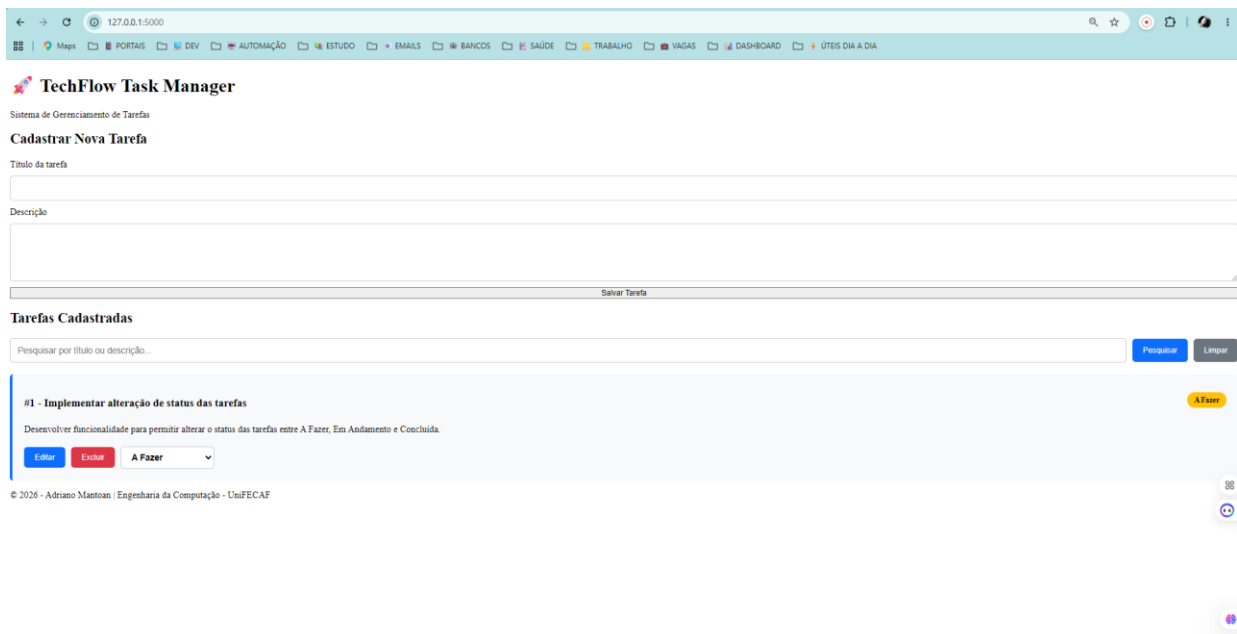
Figura 14 – Alteração para o status "Concluída"



Fonte: Elaborado pelo autor (2026).

Na sequência realizei novos testes alterando uma tarefa para o status Concluída, confirmando que a aplicação registrava corretamente o encerramento da atividade.

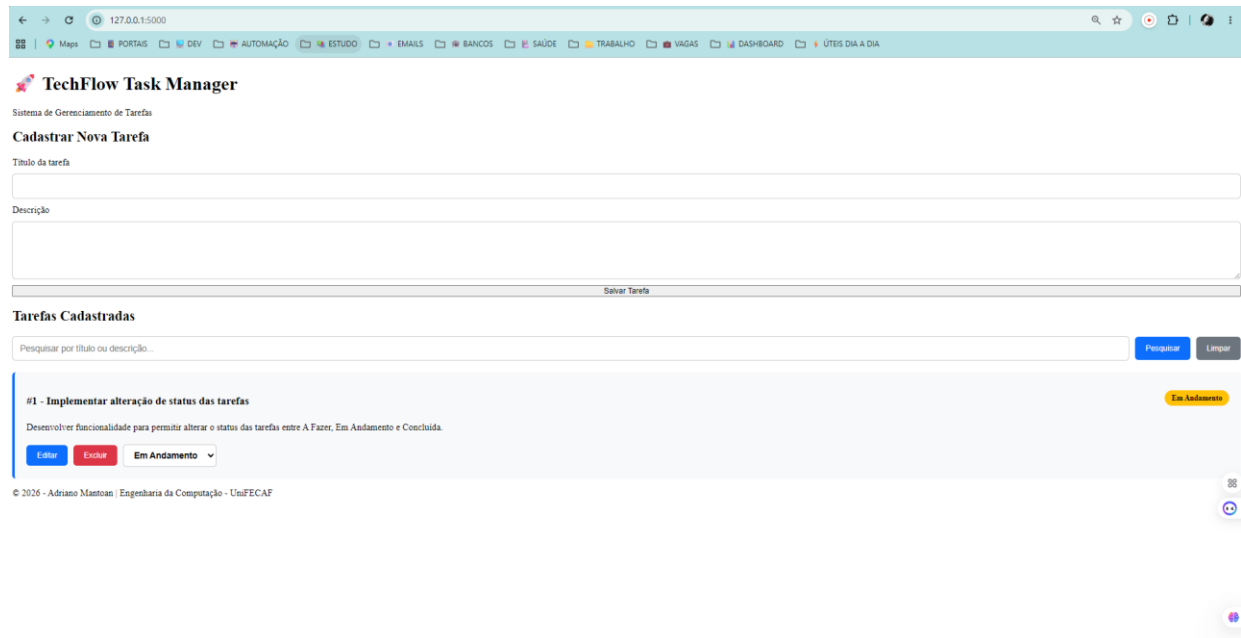
Figura 15 – Implementação da persistência utilizando SQLite



Fonte: Elaborado pelo autor (2026).

Uma das melhorias implementadas foi a utilização do banco de dados SQLite. A partir dessa etapa, todas as tarefas passaram a ser gravadas permanentemente, evitando a perda dos dados quando a aplicação fosse encerrada.

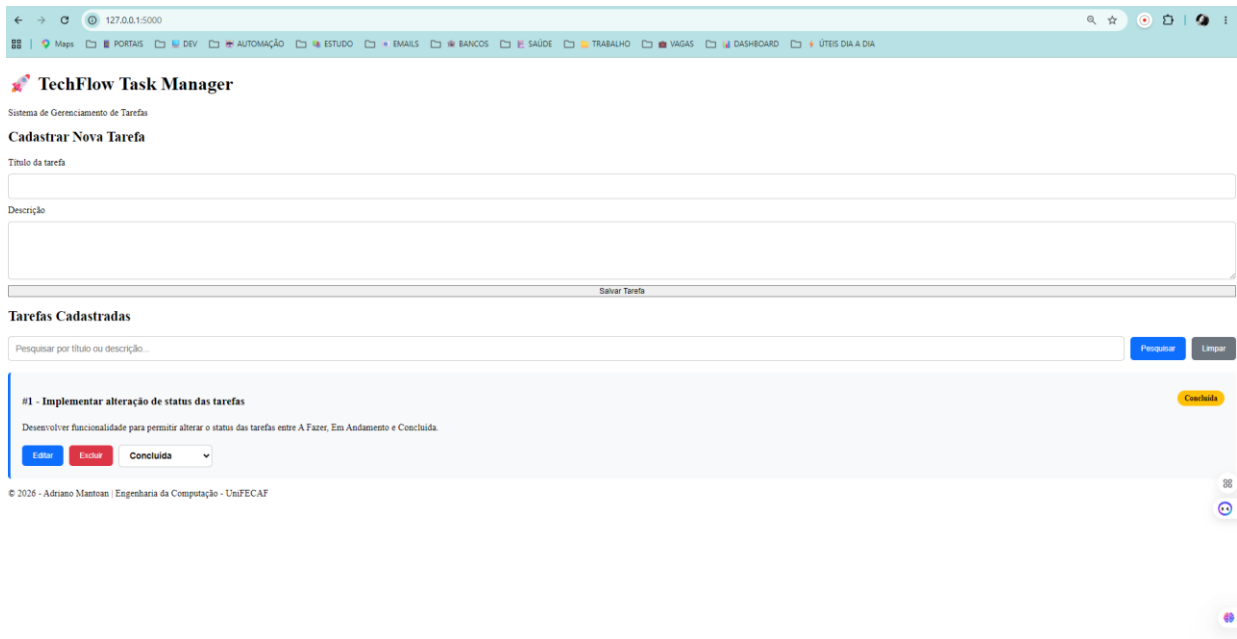
Figura 16 – Persistência validada após reinicialização



Fonte: Elaborado pelo autor (2026).

Depois da integração com o banco de dados, reiniciei a aplicação para verificar se as tarefas continuavam armazenadas. O teste confirmou que os registros permaneciam disponíveis normalmente após uma nova execução do sistema.

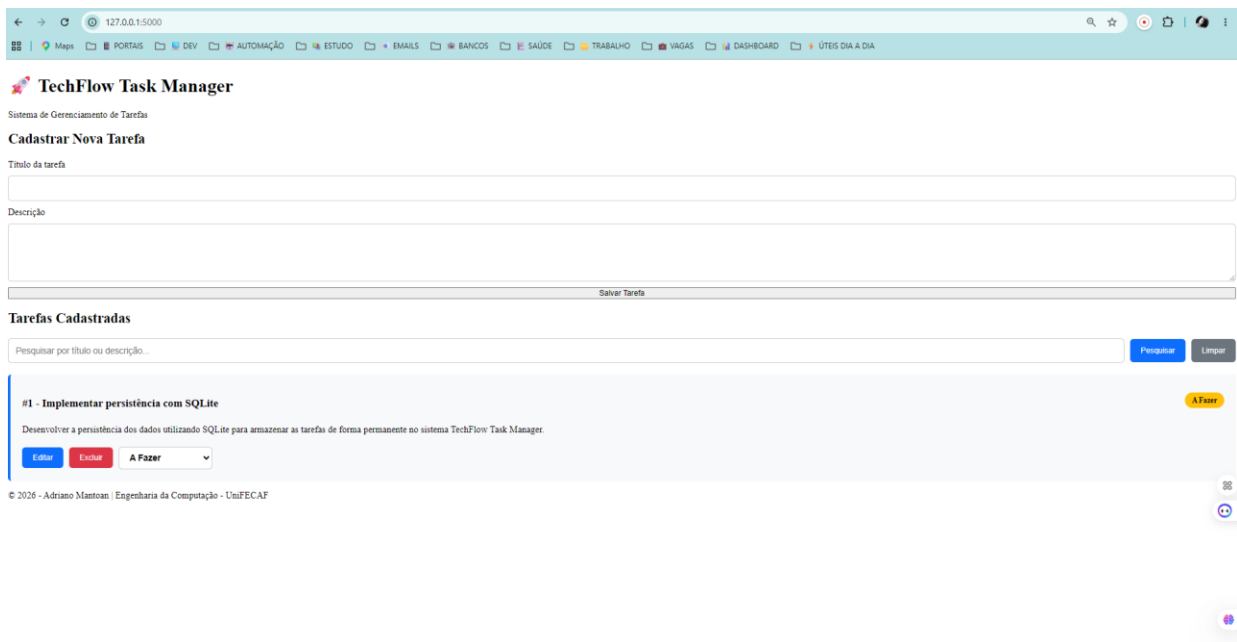
Figura 17 – Visualização da base de dados SQLite



Fonte: Elaborado pelo autor (2026).

Para validar o armazenamento das informações utilizei o software DB Browser for SQLite. Foi possível visualizar diretamente os registros gravados na tabela de tarefas, comprovando o funcionamento correto da persistência dos dados.

Figura 18 – Tela de edição de tarefas



Fonte: Elaborado pelo autor (2026).

Nesta etapa desenvolvi a tela responsável pela edição das tarefas cadastradas. O usuário passou a alterar o título e a descrição das atividades sem necessidade de criar um novo registro.

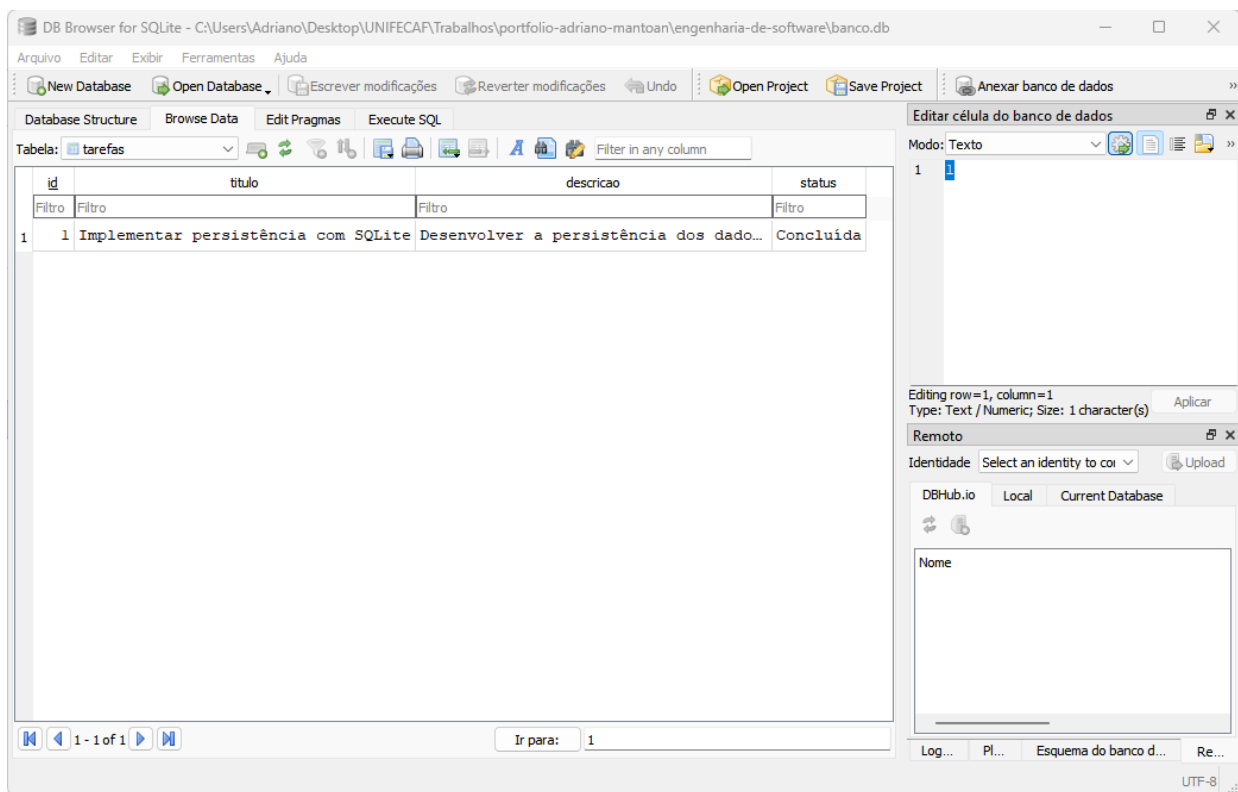
Figura 19 – Atualização das informações da tarefa

The screenshot displays the TechFlow Task Manager web application. At the top, the browser address bar shows the URL 127.0.0.1:5000. The application header includes the title 'TechFlow Task Manager' and the subtitle 'Sistema de Gerenciamento de Tarefas'. The main section is titled 'Cadastrar Nova Tarefa' and contains two input fields: 'Título da tarefa' and 'Descrição'. Below these fields is a 'Salvar Tarefa' button. Underneath the form is a section titled 'Tarefas Cadastradas' which includes a search bar labeled 'Pesquisar por título ou descrição...' and buttons for 'Pesquisar' and 'Limpar'. A list of tasks is shown below the search bar, with the first task being '#1 - Implementar persistência com SQLite'. This task has a status of 'Concluída' and a 'Concluir' button. The footer of the application shows the copyright information: '© 2026 - Adriano Mantovan | Engenharia da Computação - UniFECAF'.

Fonte: Elaborado pelo autor (2026).

Após implementar a edição, realizei testes alterando as informações de uma tarefa existente. A atualização foi realizada corretamente, mantendo a integridade dos dados armazenados no banco de dados.

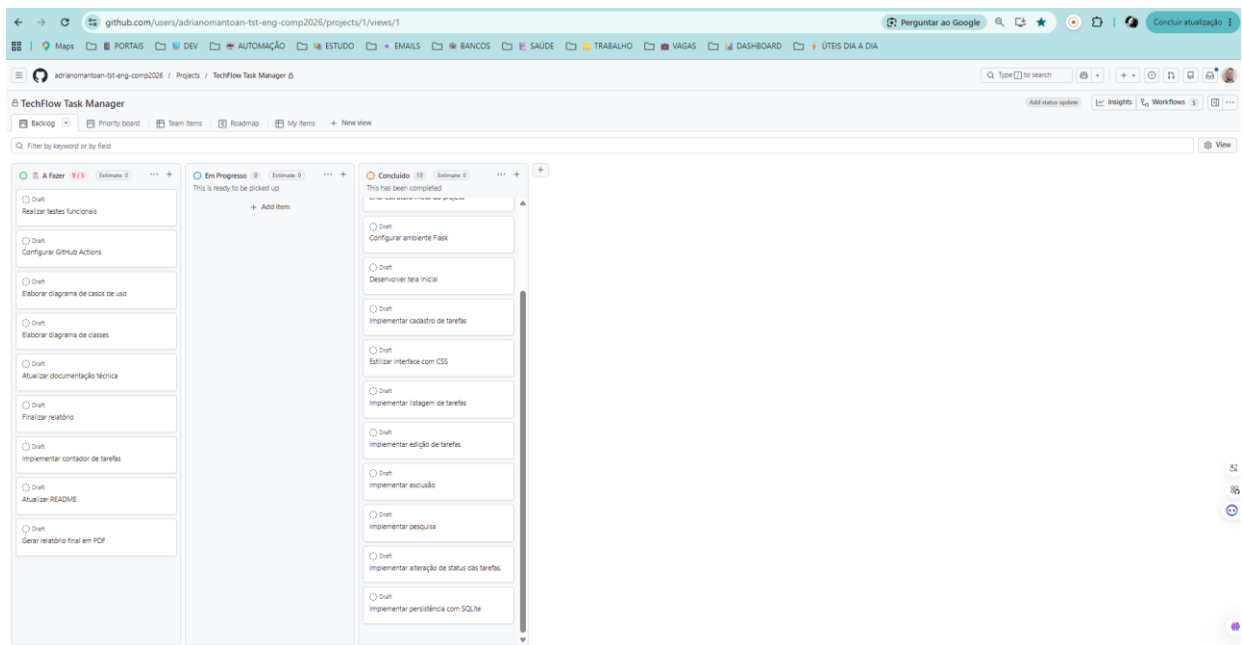
Figura 20 – Implementação da exclusão de tarefas



Fonte: Elaborado pelo autor (2026).

Nesta etapa desenvolvi a funcionalidade de exclusão de tarefas. O botão Excluir remove permanentemente o registro selecionado da base de dados, permitindo manter apenas as atividades que ainda são necessárias para o gerenciamento do projeto.

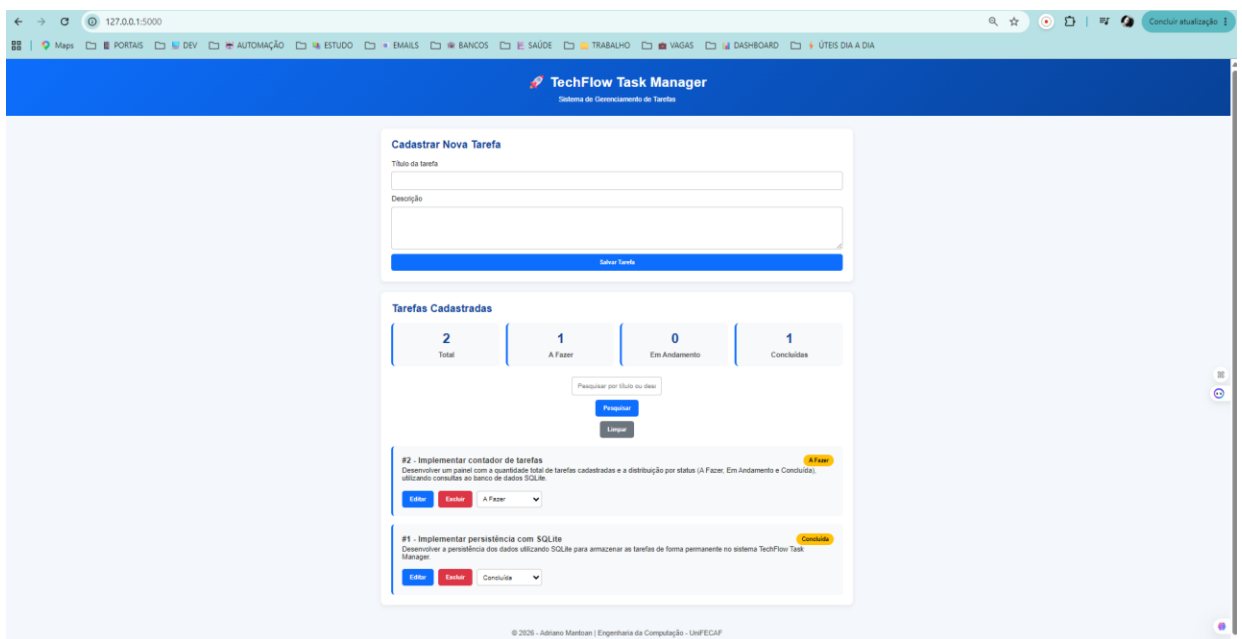
Figura 21 – Sistema sem tarefas cadastradas após exclusão



Fonte: Elaborado pelo autor (2026).

Após remover todas as tarefas cadastradas, o sistema passou a exibir uma mensagem informando que não existem registros disponíveis. Esse teste confirmou o funcionamento correto da exclusão e do tratamento da interface quando não há dados cadastrados.

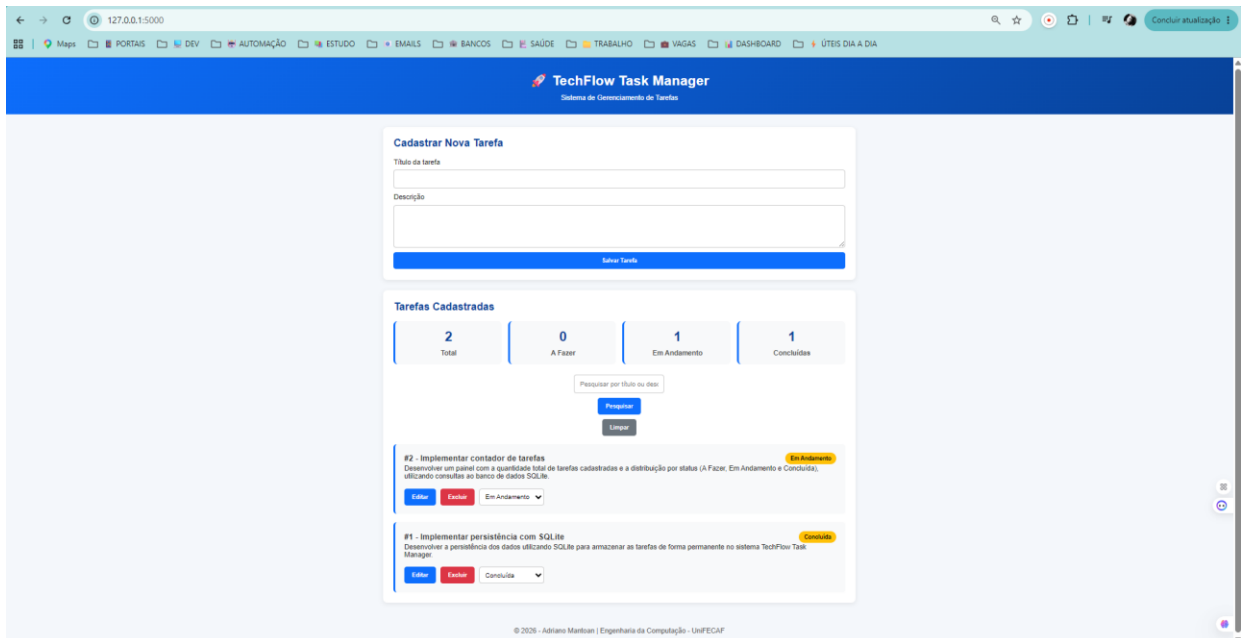
Figura 22 – Refinamento da estrutura do projeto



Fonte: Elaborado pelo autor (2026).

Após concluir as principais funcionalidades, reorganizei novamente a estrutura do projeto para facilitar sua manutenção. Revisei a disposição das pastas, dos arquivos HTML, CSS, JavaScript e demais componentes utilizados pela aplicação, deixando o código mais organizado para futuras atualizações.

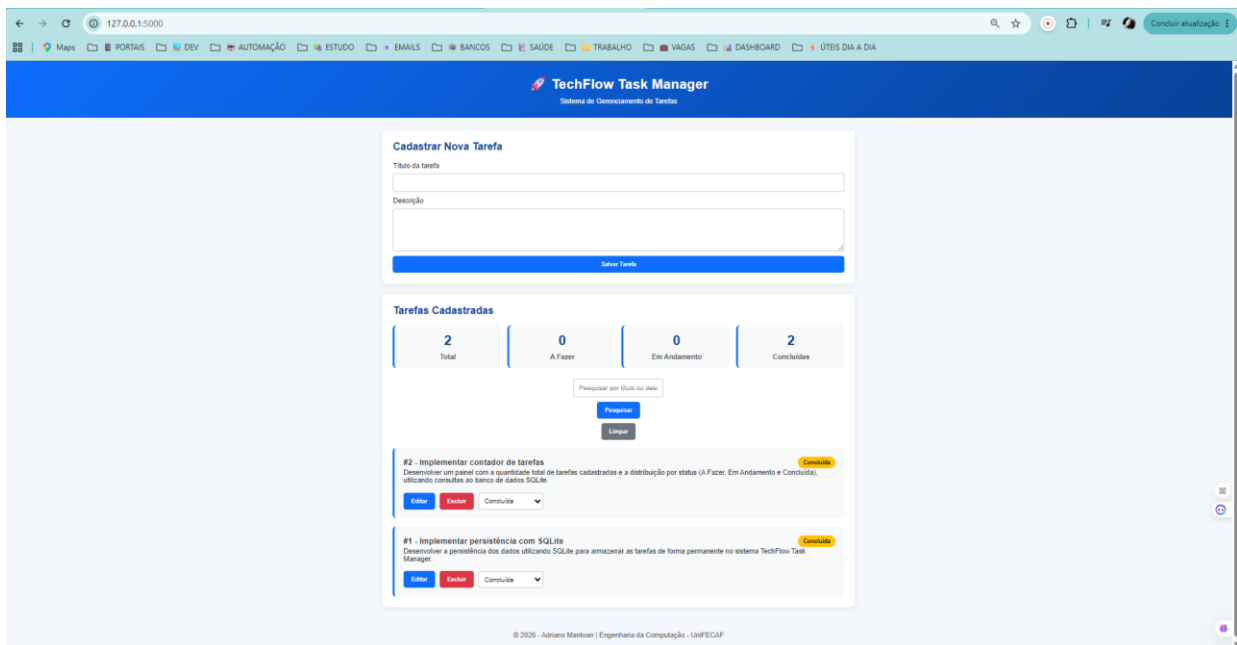
Figura 23 – Ajustes na interface inicial do sistema



Fonte: Elaborado pelo autor (2026).

Nesta etapa realizei melhorias na tela inicial do TechFlow Task Manager. Foram feitos ajustes visuais na disposição dos elementos, tornando a página mais organizada e agradável para o usuário logo no primeiro acesso.

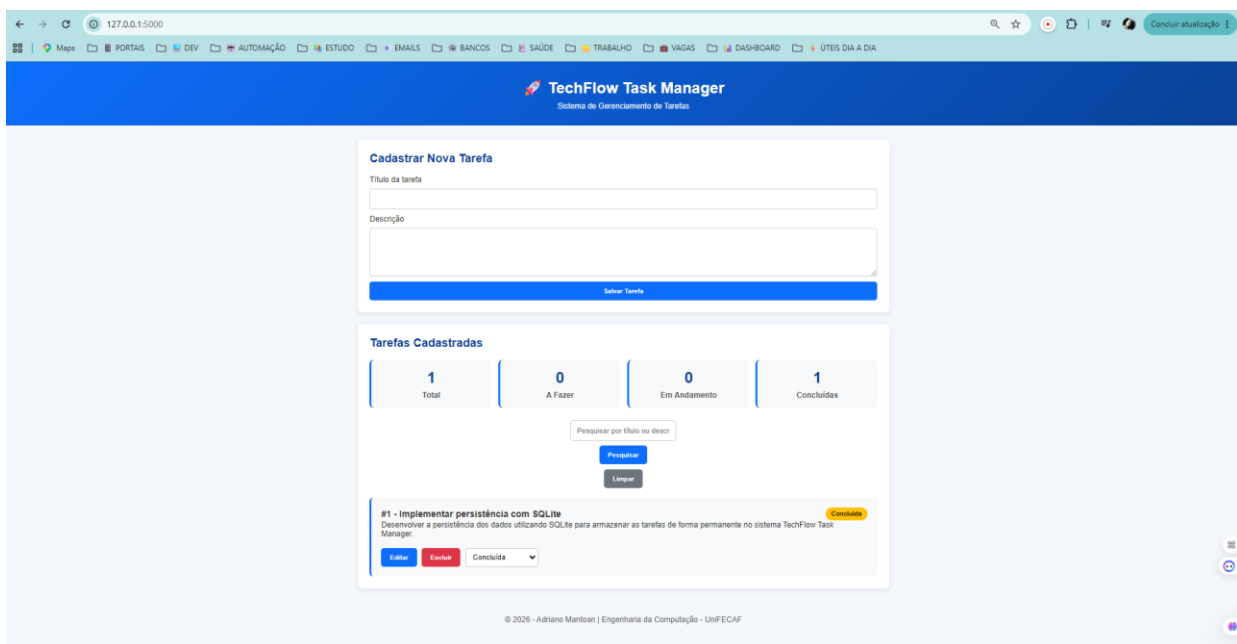
Figura 24 – Aplicação da identidade visual do projeto



Fonte: Elaborado pelo autor (2026).

Depois de concluir as funcionalidades principais, trabalhei na identidade visual do sistema. Foram aplicadas cores, botões, espaçamentos e um layout mais moderno, tornando a aplicação mais próxima de um sistema utilizado em ambiente real.

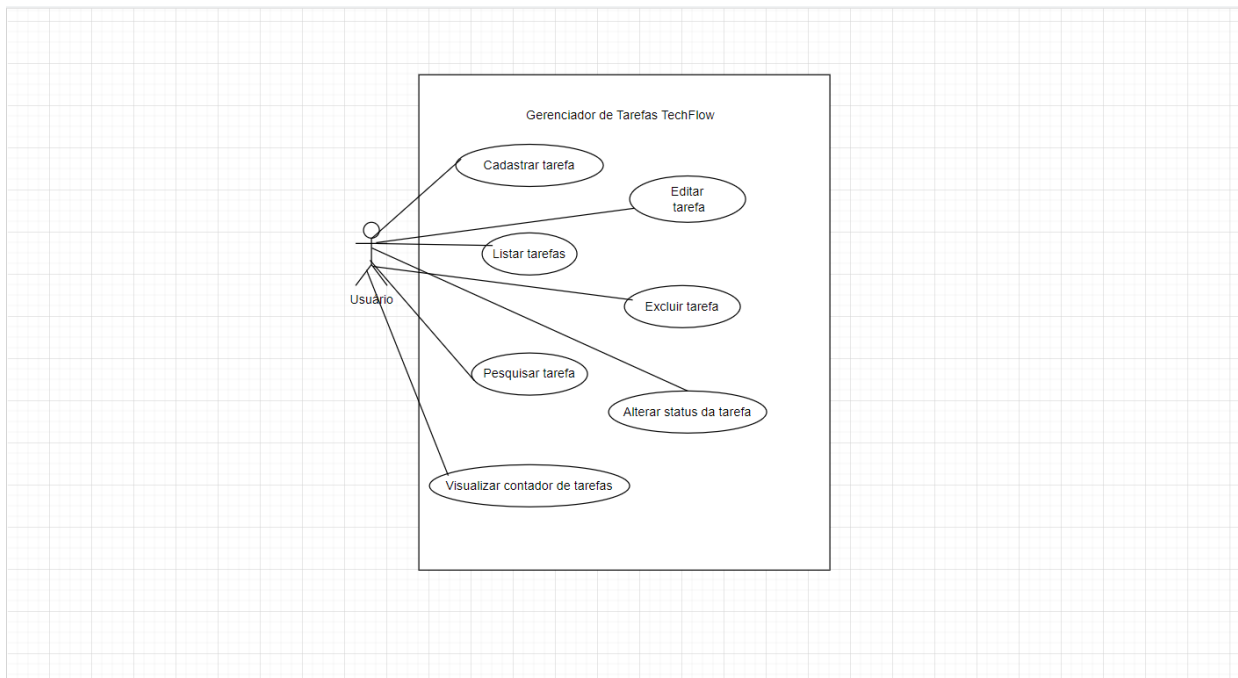
Figura 25 – Tela inicial após melhorias de layout



Fonte: Elaborado pelo autor (2026).

Com as alterações implementadas, a tela inicial passou a apresentar uma aparência mais profissional. O objetivo foi melhorar a experiência do usuário, organizando melhor as informações e facilitando a navegação dentro do sistema.

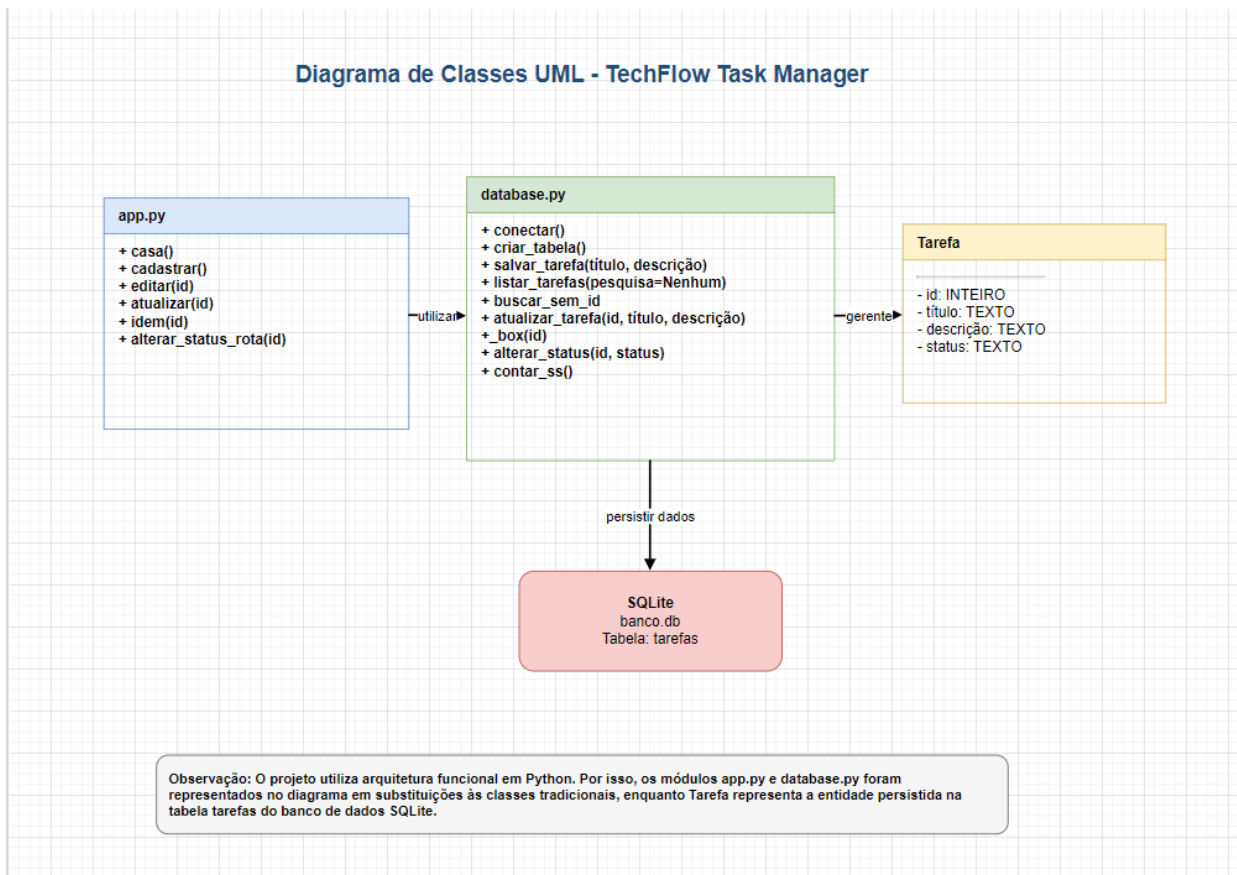
Figura 26 – Tela de edição após ajustes visuais



Fonte: Elaborado pelo autor (2026).

Também realizei melhorias na tela responsável pela edição das tarefas. Os campos permaneceram com a mesma funcionalidade, porém receberam uma organização visual mais adequada, facilitando a atualização das informações cadastradas.

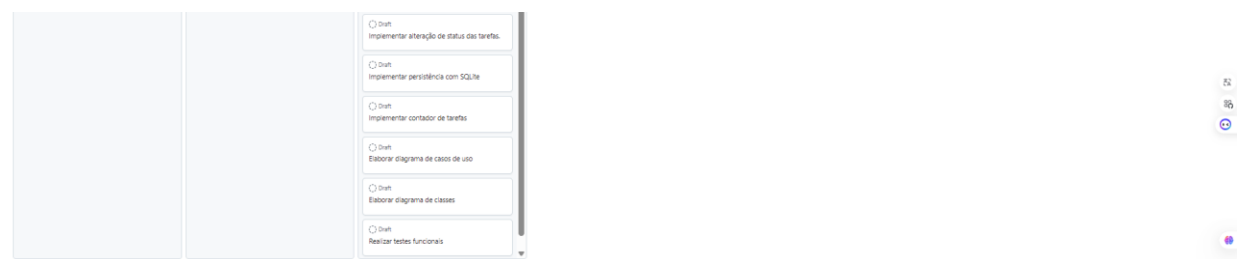
Figura 27 – Atualização completa das informações da tarefa



Fonte: Elaborado pelo autor (2026).

Nesta etapa realizei novos testes editando completamente uma tarefa já cadastrada. O sistema atualizou corretamente todas as informações, demonstrando que a funcionalidade de edição estava funcionando de forma estável.

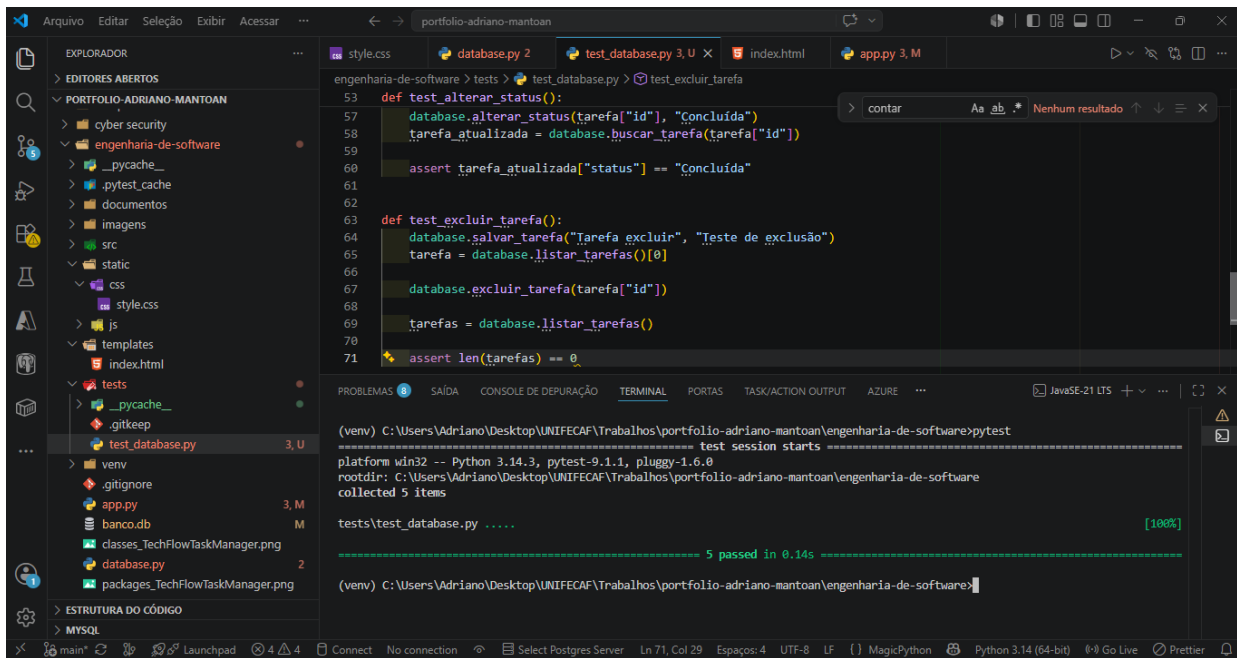
Figura 28 – Validação da funcionalidade de exclusão



Fonte: Elaborado pelo autor (2026).

Após concluir a implementação da exclusão, executei novos testes para verificar se as tarefas eram removidas corretamente da aplicação e também do banco de dados. Os resultados confirmaram o funcionamento esperado.

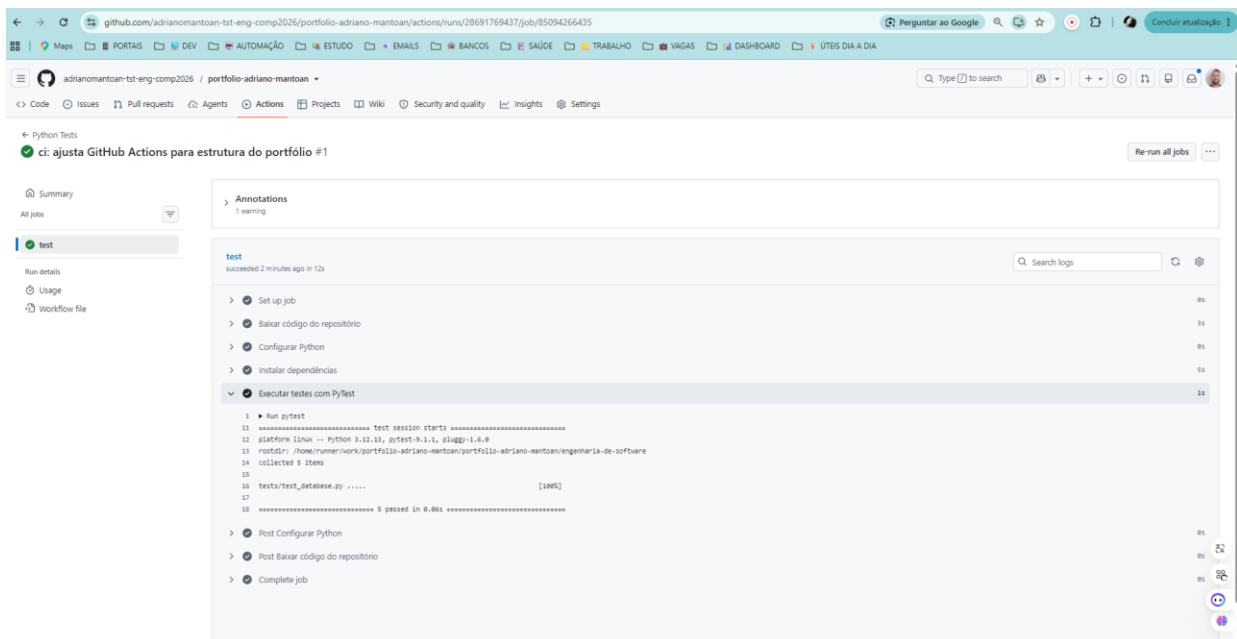
Figura 29 – Sistema após remoção de todas as tarefas



Fonte: Elaborado pelo autor (2026).

Depois da exclusão de todos os registros, a aplicação passou a exibir uma mensagem informando que não existem tarefas cadastradas. Essa validação garantiu que o sistema trata corretamente situações em que a base de dados está vazia.

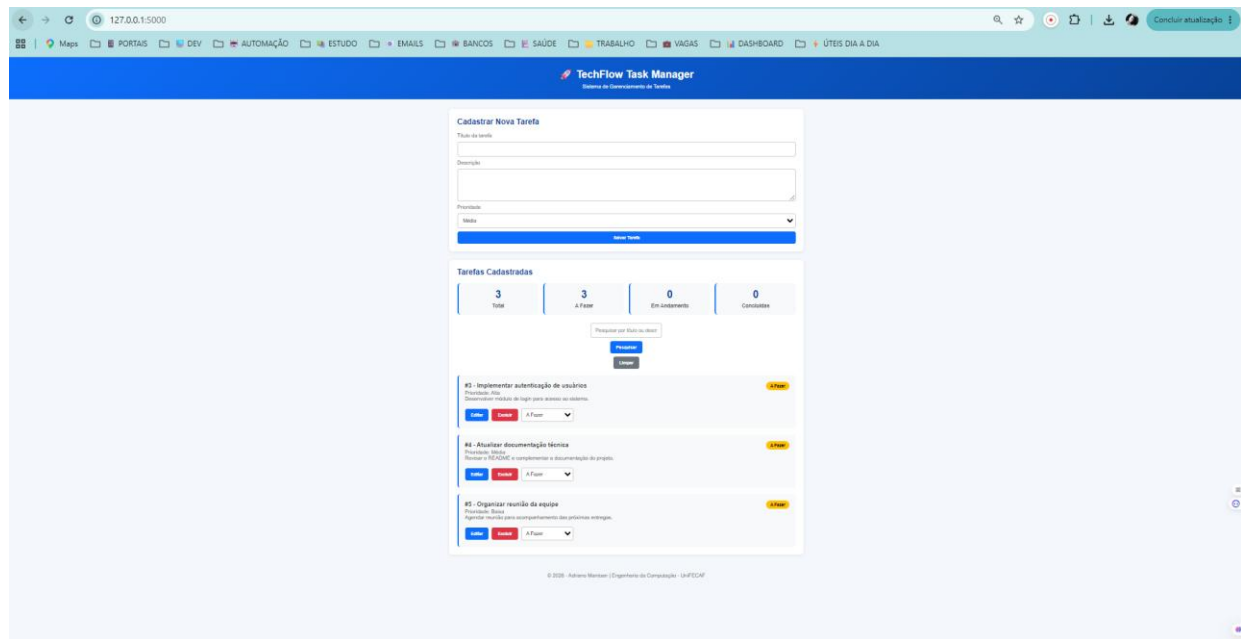
Figura 30 – Pesquisa sem resultados encontrados



Fonte: Elaborado pelo autor (2026).

Também realizei testes utilizando o campo de pesquisa quando não existiam tarefas compatíveis com o termo informado. O sistema apresentou uma mensagem adequada, indicando que nenhum registro foi localizado.

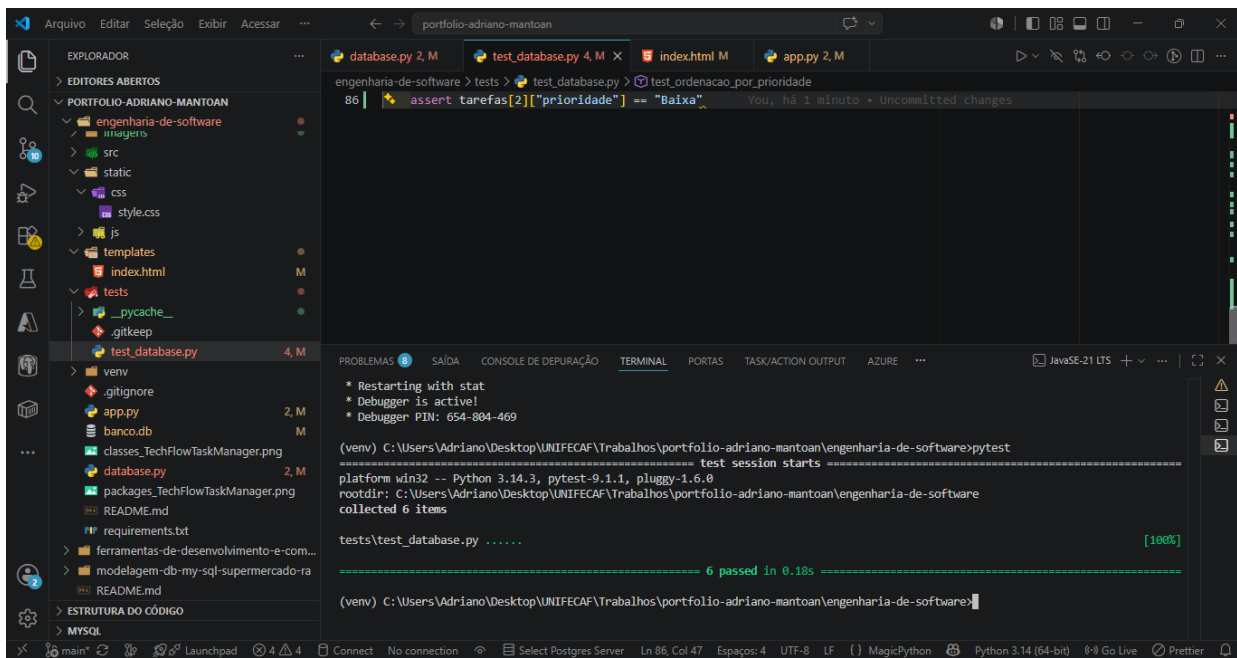
Figura 31 – Revisão final da aplicação



Fonte: Elaborado pelo autor (2026).

Com todas as funcionalidades implementadas, realizei uma revisão geral do sistema, verificando o funcionamento do cadastro, edição, exclusão, pesquisa, alteração de status e persistência dos dados. Essa etapa foi importante para identificar pequenos ajustes antes da finalização do projeto.

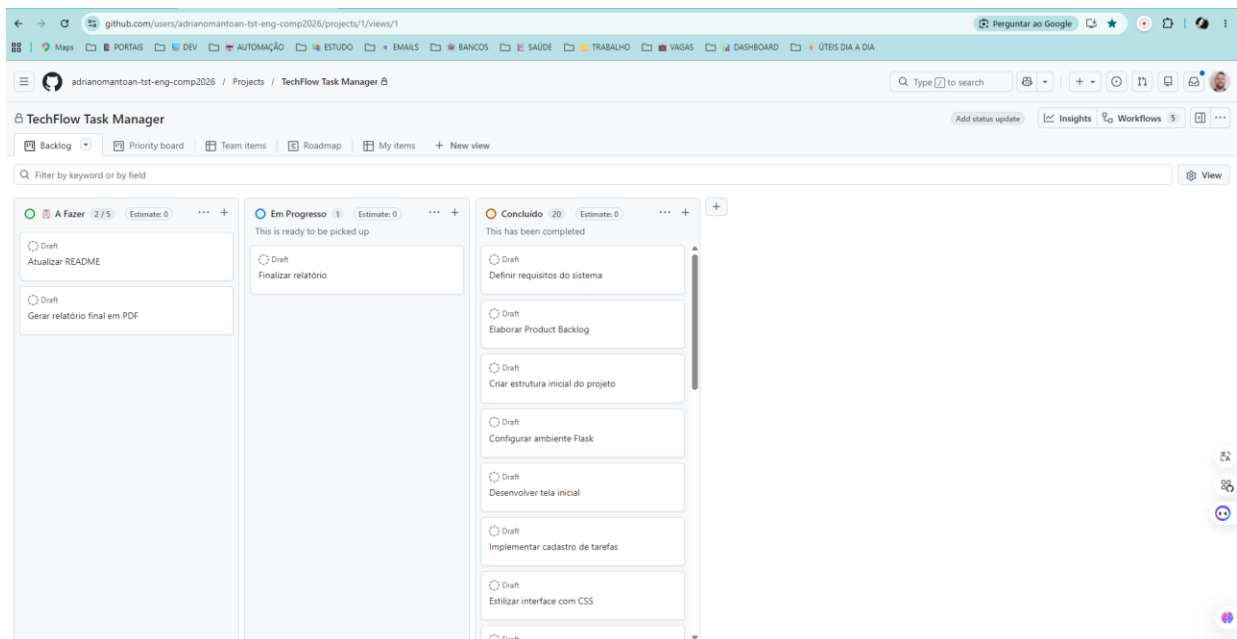
Figura 32 – Sistema concluído



Fonte: Elaborado pelo autor (2026).

Nesta etapa o desenvolvimento foi considerado concluído. Todas as funcionalidades previstas no Product Backlog estavam implementadas e funcionando conforme o planejado, atendendo aos objetivos definidos para o projeto.

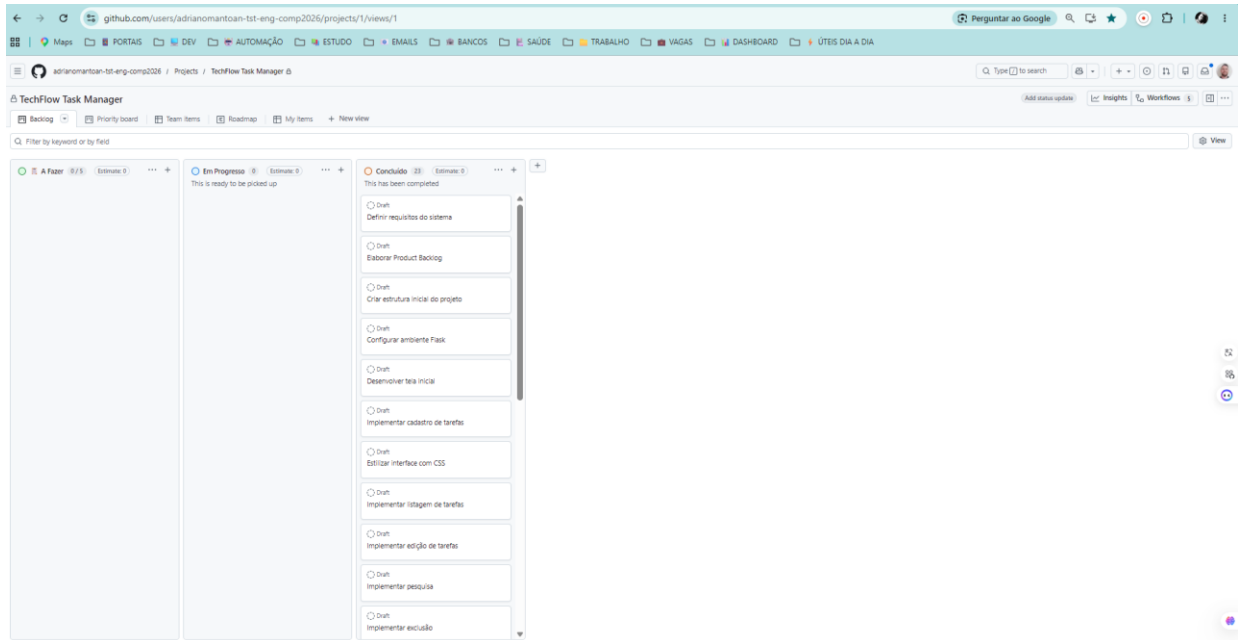
Figura 33 – Organização final do projeto



Fonte: Elaborado pelo autor (2026).

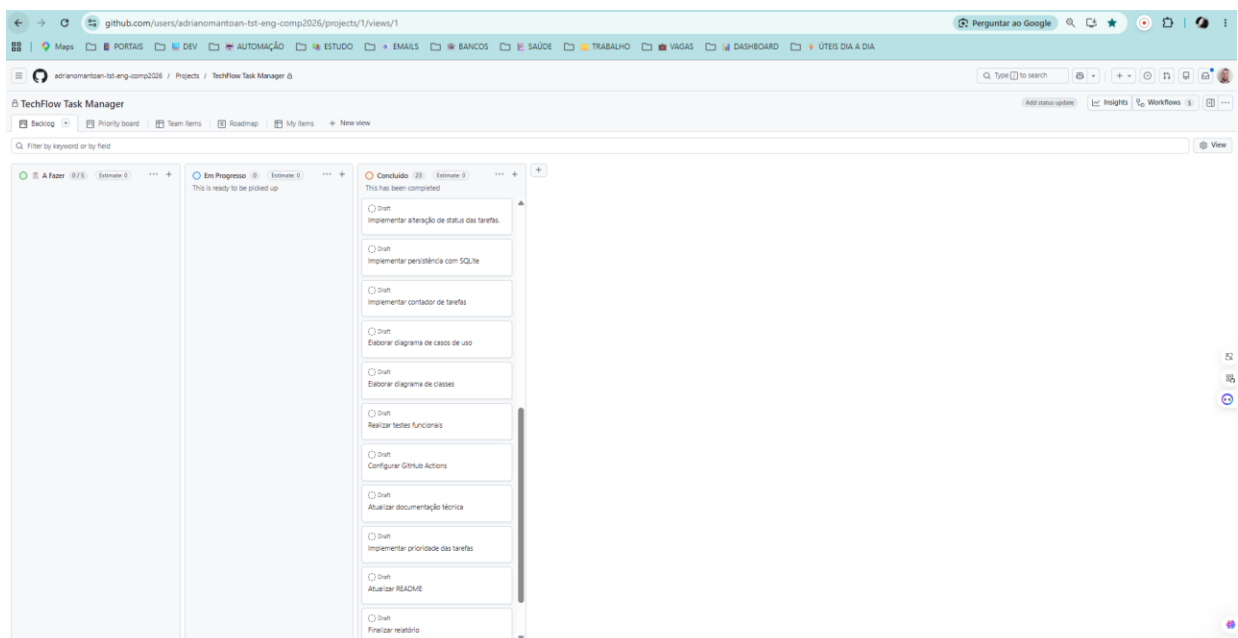
Após finalizar o desenvolvimento, organizei todos os arquivos do projeto, incluindo códigos-fonte, documentação e estrutura de diretórios. Essa organização facilita futuras manutenções e permite que o projeto seja compreendido com maior facilidade.

Figura 34.1 – Início do quadro Kanban com as primeiras atividades concluídas do projeto TechFlow Task Manager.



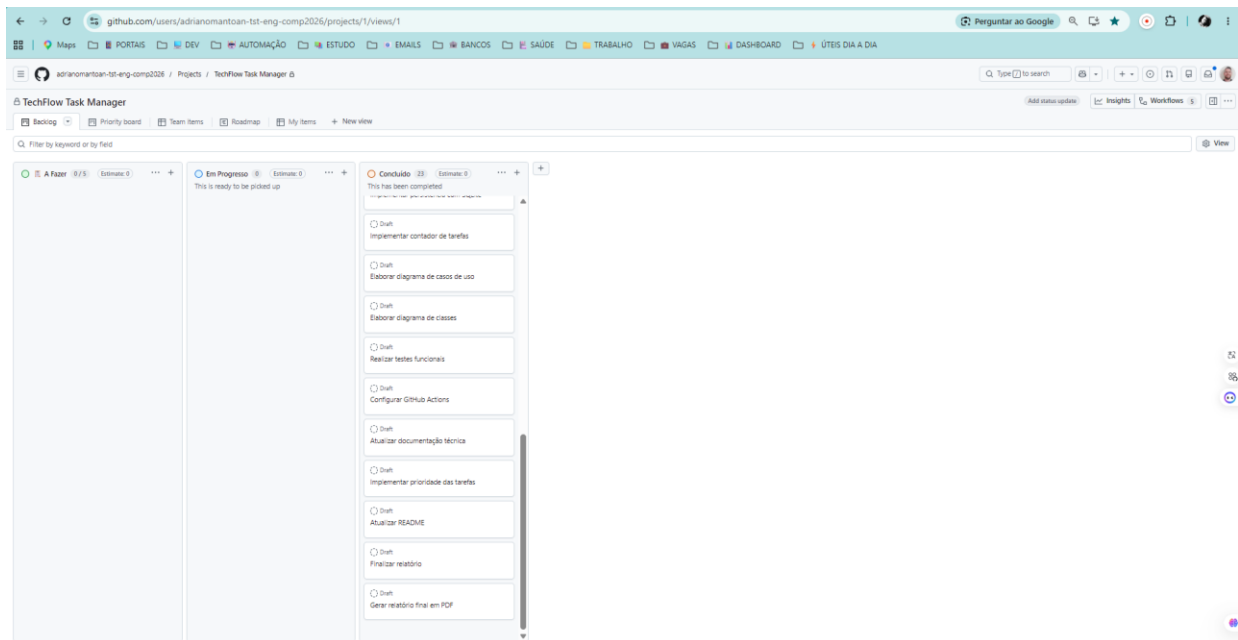
Fonte: Elaborado pelo autor (2026).

Figura 34.2 – Parte central do quadro Kanban contendo a continuidade das atividades concluídas do projeto.



Fonte: Elaborado pelo autor (2026).

Figura 34.3 – Parte final do quadro Kanban apresentando a conclusão de todas as atividades planejadas para o projeto TechFlow Task Manager.



Fonte: Elaborado pelo autor (2026).

A conferência final do quadro Kanban permitiu verificar que todas as atividades previstas no Product Backlog foram concluídas com sucesso. Dessa forma, foi possível encerrar o desenvolvimento do sistema TechFlow Task Manager com todas as funcionalidades planejadas implementadas, testadas e documentadas.

5. Dificuldades Encontradas

Durante o desenvolvimento deste projeto encontrei algumas dificuldades que fizeram parte do meu processo de aprendizagem. No início, precisei compreender melhor como estruturar uma aplicação utilizando o framework Flask, principalmente em relação à organização das pastas, das rotas e da integração entre os arquivos Python, HTML e CSS. Como ainda estou adquirindo experiência na área de desenvolvimento de software, foi necessário pesquisar bastante e realizar diversos testes até entender como cada parte da aplicação funcionava.

Outro desafio foi implementar a persistência dos dados utilizando o banco de dados SQLite. Aprendi como criar a conexão com o banco, armazenar as informações das tarefas e atualizar os registros sempre que uma alteração era realizada. Em alguns momentos ocorreram erros de programação que exigiram revisar o código, identificar a origem do problema e realizar novos testes até encontrar a solução adequada.

Também encontrei dificuldades durante a implementação da funcionalidade de alteração do status das tarefas. Foi necessário ajustar a lógica da aplicação para que uma atividade pudesse ser alterada corretamente entre os estados "A Fazer", "Em Andamento" e "Concluída", mantendo as informações atualizadas tanto na interface quanto no banco de dados.

Outro aprendizado importante foi utilizar o GitHub Projects para organizar todas as atividades do projeto por meio da metodologia Kanban. No início tive dificuldade para compreender como estruturar o quadro e movimentar os cartões conforme a evolução das tarefas. Com a prática, consegui utilizar essa ferramenta como apoio para acompanhar todo o desenvolvimento do sistema.

A configuração dos testes automatizados utilizando PyTest e da Integração Contínua com GitHub Actions também representou um desafio, pois foi meu primeiro contato com essas ferramentas. Apesar das dificuldades iniciais, consegui compreender seu funcionamento e verificar a importância da automação dos testes durante o desenvolvimento de software.

Ao finalizar este projeto percebo que cada dificuldade enfrentada contribuiu diretamente para ampliar meus conhecimentos. Mais do que desenvolver uma aplicação funcional, aprendi a pesquisar soluções, corrigir erros, testar funcionalidades e compreender melhor todo o processo de desenvolvimento de software.

6. Análise dos Resultados

Ao concluir o desenvolvimento do TechFlow Task Manager, pude verificar que os objetivos definidos no início do projeto foram alcançados. O sistema desenvolvido permite cadastrar, editar, excluir e pesquisar tarefas, além de controlar o andamento das atividades por meio dos status "A Fazer", "Em Andamento" e "Concluída". Todas

essas funcionalidades foram implementadas e testadas durante o desenvolvimento da aplicação.

Outro resultado importante foi a utilização do banco de dados SQLite para garantir a persistência das informações. Diferentemente das primeiras versões da aplicação, em que os dados eram perdidos ao encerrar o sistema, a implementação do banco de dados permitiu armazenar permanentemente todas as tarefas cadastradas.

A utilização do GitHub Projects contribuiu significativamente para a organização das atividades. O quadro Kanban permitiu acompanhar cada etapa do projeto, facilitando o gerenciamento das tarefas e proporcionando uma visão clara da evolução do desenvolvimento desde o planejamento até a conclusão.

Os testes automatizados desenvolvidos com PyTest demonstraram que as funcionalidades implementadas estavam operando corretamente. Além disso, a configuração do GitHub Actions possibilitou automatizar a execução desses testes sempre que novas alterações eram enviadas ao repositório, proporcionando maior confiabilidade ao projeto.

Outro aspecto importante foi a elaboração dos Diagramas de Casos de Uso e de Classes UML. Esses diagramas facilitaram a compreensão da estrutura da aplicação e contribuíram para organizar melhor a implementação das funcionalidades antes mesmo do desenvolvimento do código.

Ao analisar o resultado final, considero que o projeto atingiu o objetivo proposto pela disciplina, permitindo aplicar na prática diversos conceitos estudados em Engenharia de Software, como levantamento de requisitos, planejamento das atividades, desenvolvimento incremental, modelagem UML, persistência de dados, testes automatizados, integração contínua e controle de versões utilizando Git e GitHub.

Além do resultado técnico obtido, acredito que o maior aprendizado foi compreender que o desenvolvimento de software envolve muito mais do que apenas escrever código. Planejamento, organização, documentação, testes, validação e melhoria contínua fazem parte de todo o processo e foram fundamentais para a conclusão deste projeto.

7. Conclusão

O desenvolvimento do TechFlow Task Manager representou uma oportunidade de colocar em prática diversos conceitos estudados durante a disciplina de Engenharia de Software. Ao longo do projeto consegui compreender que a construção de um sistema envolve muito mais do que programação, exigindo planejamento, organização das atividades, documentação, testes e acompanhamento constante da evolução do desenvolvimento.

Durante a execução da atividade desenvolvi uma aplicação capaz de realizar o gerenciamento de tarefas, implementando funcionalidades como cadastro, edição, exclusão, pesquisa, alteração de status, persistência em banco de dados SQLite, testes automatizados com PyTest e integração contínua utilizando GitHub Actions. Além disso, utilizei o GitHub Projects para acompanhar todas as etapas do projeto por meio da metodologia Kanban e elaborei os diagramas UML para representar a estrutura e o funcionamento da aplicação.

Ao comparar meus conhecimentos no início da disciplina com os adquiridos ao longo deste trabalho, percebo uma evolução significativa. Antes possuía pouco contato com ferramentas como Flask, SQLite, GitHub Projects, PyTest e GitHub Actions. Durante o desenvolvimento consegui compreender melhor a finalidade de cada uma delas e sua importância dentro do ciclo de desenvolvimento de software.

Considero que este projeto contribuiu de forma significativa para minha formação acadêmica, pois permitiu aplicar na prática conceitos que antes eram apenas teóricos. Além disso, a experiência adquirida servirá como base para os próximos projetos da graduação e também para minha evolução profissional, ampliando meus conhecimentos na área de desenvolvimento de software e fortalecendo minha integração entre Tecnologia da Informação e minha experiência profissional em Segurança do Trabalho.

8. Referências

Python. Python Documentation.

Flask. Flask Documentation.

SQLite. SQLite Documentation.

Git. Git Documentation.

GitHub. GitHub Documentation.

PyTest. PyTest Documentation.

Visual Studio Code. Visual Studio Code Documentation.

draw.io. draw.io Documentation.

Repositório do Projeto

O código-fonte, a documentação e os arquivos desenvolvidos durante este projeto encontram-se disponíveis no repositório GitHub:

Portfólio:

<https://github.com/adrianomantoan-tst-eng-comp2026/portfolio-adriano-mantoan>

Projeto Engenharia de Software:

<https://github.com/adrianomantoan-tst-eng-comp2026/portfolio-adriano-mantoan/tree/main/engenharia-de-software>